Status Completed

Depotube project functional specification

Program manager/architect: John C Graham

Introduction

1.0 Project Overview and Feature Description

What is depotube?

depotube is a cloud-based deposition management platform designed specifically for law firms and legal departments. The platform solves three critical challenges in modern litigation:

1. **Storage and Organization** - Deposition videos are large files (often 50-100GB for a full-day deposition) that are expensive to store and difficult to organize across multiple cases and clients.
2. **Searchability** - Video depositions are time-consuming to review. Without transcription, attorneys must manually scrub through hours of footage to find specific testimony, wasting billable hours.
3. **Legal Analysis** - Even with transcripts, attorneys spend significant time identifying relevant case law that supports or contradicts witness testimony.

depotube addresses these challenges by providing:

- **Secure cloud storage** organized by client and case
- **Automatic transcription** of all deposition video and audio into searchable text
- **AI-powered legal summaries** that identify key testimony and cite relevant case law, reducing review time from hours to minutes

1.1 Document Structure

This functional specification is organized into the following sections:

Section 1: Introduction

Section 2: High-level feature and architectural overview

- **2.1 Project Overview and Feature Description**
- **2.2 Understanding AI Summaries**
- **2.3 Virtual Architecture and Data Model**
- **2.4 Typical Workflows**
- **2.5 Document Scope and Forward Compatibility**

Section 3: Authentication - How to authenticate API requests using OAuth 2.0, including permission models for different user roles (admin, attorney, paralegal) and token-based access control.

Section 4: Client Management - Creating, retrieving, updating, and deleting client records. Each client represents a person or entity involved in litigation.

Section 5: Deposition Upload - Uploading pre-recorded deposition video files from a web browser, including multipart upload requests and polling for upload progress.

Section 6: Live Deposition Streaming - Streaming live depositions using WebRTC and the depotube JavaScript SDK, including starting streams, monitoring stream status, and ending streams.

Section 7: Automatic Transcription - How the platform converts deposition audio to searchable text, including transcription status polling, estimated completion times, and retrieving completed transcripts.

Section 8: AI Summary Generation - Requesting AI-powered summaries with case law citations, polling for summary completion, and retrieving structured summary data.

Section 9: Retrieving Depositions and Assets - Listing all depositions for a client, retrieving individual deposition metadata, and downloading videos, transcripts, and summaries.

Section 10: Delete Operations - Deleting depositions and clients, including cascade behavior, canceling in-progress uploads, and audit trail implications.

Section 11: Audit Logs - Retrieving compliance audit logs showing who accessed or modified what data and when.

Section 12: Error Handling - Standard error response format, common HTTP status codes, and rate limiting behavior.

Section 13: Appendix - Technical specifications including video format requirements, file size limits, transcription time estimates, and webhook configuration.

2.1 Project Overview and Core Features

For Law Firms:

- Upload completed deposition videos (MP4, MOV, WebM formats up to 100GB)
- Stream live depositions directly from the courtroom or conference room using any modern web browser
- Automatic speech-to-text transcription with speaker identification and timestamps
- Access completed transcripts within 15 minutes of upload (for a 1-hour deposition)
- Generate AI summaries with case law citations formatted in Bluebook style
- Organize all depositions by client with role-based access (admin, attorney, paralegal)
- Audit trail of all access and changes for compliance purposes

For Software Developers:

- RESTful API accessible from web browsers or server-side applications
- JavaScript SDK for browser-based WebRTC streaming (no media server infrastructure required)
- Polling endpoints for real-time upload and transcription progress
- Structured JSON responses for all deposition metadata, transcripts, and summaries

- Webhook support for asynchronous notifications (optional)

2.2 Understanding AI Summaries - The Core Value Proposition

What AI Summaries Provide

A depotube AI summary is not simply a shorter version of the transcript. It's a legal analysis document that performs three critical functions:

1. Key Testimony Extraction The AI identifies the most legally significant statements from hours of testimony, eliminating the need for attorneys to read entire transcripts. For a 4-hour deposition transcript (approximately 40,000 words), the summary highlights the 20-30 critical statements that matter for the case.

2. Credibility Analysis The AI flags internal contradictions, inconsistencies with known facts, and hedging language (e.g., "I think," "maybe," "I'm not sure"). This helps attorneys identify weak points for cross-examination or impeachment.

3. Case Law Citation The AI searches legal databases and automatically cites relevant precedents with proper Bluebook formatting. For example, if a witness testifies about informed consent in a medical malpractice case, the summary might cite *Canterbury v. Spence, 464 F.2d 772 (D.C. Cir. 1972)* with a brief explanation of relevance and the specific holding.

Time Savings Example

Traditional workflow (without depotube):

- Review 4-hour deposition transcript: 2-3 hours
- Manual case law research: 3-4 hours
- Draft summary for case file: 1-2 hours
- **Total: 6-9 billable hours**

With depotube AI summary:

- Generate AI summary: 10 minutes (automated)
- Attorney review and refinement: 30-45 minutes
- **Total: ~1 billable hour**

Summary Output Format

AI summaries are structured documents containing:

- **Executive Overview** - 2-3 paragraph synopsis of the entire deposition
- **Key Facts Admitted** - Bullet points of favorable testimony
- **Contradictions and Weaknesses** - Areas for impeachment or further discovery
- **Legal Issues Identified** - Potential claims, defenses, or procedural problems
- **Case Law Citations** - Relevant precedents with Bluebook format and explanatory parentheticals
- **Transcript Cross-References** - Every summary point links to specific transcript page/line numbers

Summaries can be exported as PDF for inclusion in case management systems or court filings.

2.3 Virtual Architecture and Data Model

The Client Silo Concept

depotube organizes all data into **client silos**. Each silo is a secure, isolated container that holds:

- **Client metadata** (name, case number, matter description)
- **All depositions** associated with that client (video files, transcripts, summaries)
- **Access permissions** defining which users can view, upload, or delete content
- **Audit logs** tracking all actions performed within the silo

Data isolation is absolute. Users with access to "Client A" cannot see, search, or access any data belonging to "Client B," even if they work at the same law firm. This ensures attorney-client privilege and confidentiality across multiple cases.

User Roles and Permissions

depotube supports three user roles with different permission levels:

Administrator

- Create, modify, and delete clients (silos)
- Grant or revoke user access to specific client silos
- Upload and stream depositions

- Request transcription and AI summaries
- Download all assets (video, transcripts, summaries)
- Delete depositions
- View audit logs

Attorney

- Upload and stream depositions to client silos they have access to
- Request transcription and AI summaries
- Download all assets
- View audit logs for clients they have access to
- **Cannot** create or delete clients
- **Cannot** modify user permissions

Paralegal

- Upload depositions to client silos they have access to
- Download transcripts and summaries
- **Cannot** stream depositions
- **Cannot** request AI summaries (attorney review required)
- **Cannot** delete depositions
- View limited audit log information

Available Actions (in Plain English)

Available Actions

Users can perform actions at two levels: **top-level account actions** (done via Admin user) and **within-silo actions**.

Top-Level Account Actions (Admin Only)

These actions operate at the law firm account level, outside any specific client silo:

1. **Create a new client silo** - Establish a new case workspace for a client
2. **Delete a client silo** - Permanently remove an entire client and all associated depositions (requires confirmation)
3. **Manage user permissions** - Grant or revoke user access to specific client silos
4. **View cross-silo audit logs** - Review compliance records across all clients

Within-Silo Actions

Within each client silo, authorized users can perform the following actions (subject to role permissions):

1. **Upload a deposition** - Transfer a pre-recorded video file from browser to cloud storage, triggering automatic transcription (Attorney, Admin)
2. **Stream a live deposition** - Record a deposition in real-time using WebRTC, with simultaneous transcription processing (Attorney, Admin)
3. **Monitor upload/transcription progress** - Check status of in-progress uploads and transcription jobs (All roles)
4. **List all depositions** - View all depositions within this client silo, sorted by date or status (All roles)
5. **Retrieve a specific deposition** - Get metadata, status, and available assets for a single deposition (All roles)
6. **Download video** - Retrieve the original deposition video file (Attorney, Admin)
7. **Download transcript** - Retrieve the searchable text transcript with timestamps and speaker identification (All roles)
8. **Request AI summary** - Initiate case law analysis and summary generation (Attorney, Admin only)
9. **Download AI summary** - Retrieve structured summary with case law citations in Bluebook format (All roles)
10. **Delete a deposition** - Permanently remove a deposition and all associated assets (Admin only, leaves audit trail)
11. **View silo audit logs** - Review compliance records of all access and modifications within this client silo (Attorney, Admin)

Diagrams

[DIAGRAM 1: Client Silo Architecture]

```
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                                LAW FIRM ACCOUNT                                      │
│                                                                                      │
│  ┌──────────────────────────┐  ┌──────────────────────────┐  ┌──────────────────┐  │
│  │ Client Silo A            │  │ Client Silo B            │  │ Silo C           │  │
```

```
│  │                          │  │                          │  │                 │ │
│  │ Smith, John              │  │ Acme Corp v.             │  │ Doe v.          │ │
│  │ CV-2026-12345            │  │ State of CA              │  │ City      │ │
│  │                      │  │                          │  │               │ │
│  │ • Deposition 1           │  │ • Deposition 1           │  │ • Dep 1         │ │
│  │ • Deposition 2           │  │ • Deposition 2           │  │ • Dep 2         │ │
│  │ • Deposition 3           │  │ • Deposition 3           │  │                 │ │
│  │                          │  │ • Deposition 4           │  │                 │ │
│  └──────────────────────────┘  └──────────────────────────┘  └─────────────────┘ │
│                                  │
│  Isolated storage - no cross-silo access              │
└──────────────────────────────────────────────────────────────────────────────────────
────┘
```

[DIAGRAM 2: User Access Permissions]

```
┌──────────────────┐       ┌──────────────────────┐
│ Admin User   │──────────>│  Client Silo A  │ (Full Access)
└──────────────────┘    │  └──────────────────────┘
              │
              └──> ┌──────────────────────┐
                │  Client Silo B  │ (Full Access)
                └──────────────────────┘


┌──────────────────┐       ┌──────────────────────┐
```

```
│ Attorney 1  │──────────>│ Client Silo A  │ (Upload, Download, Summarize)
└──────────────┘          └────────────────┘


┌──────────────┐          ┌────────────────┐
│ Paralegal 1 │──────────>│ Client Silo A  │ (Upload, Download Only)
└──────────────┘          └────────────────┘


                 ┌────────────────┐
                 │ Client Silo B  │ (No Access)
                 └────────────────┘
```

[DIAGRAM 3: Deposition Processing Flow]

```
1. UPLOAD/STREAM        2. TRANSCRIBE        3. SUMMARIZE (Optional)

┌──────────────┐        ┌──────────────┐        ┌──────────────┐
│              │        │              │        │              │
│ Video File  │──────────>│ Speech-to-  │──────────>│ AI Analysis  │
│ (MP4/WebRTC) │        │ Text Engine  │        │ + Case Law   │
│              │        │              │        │              │
└──────────────┘        └──────────────┘        └──────────────┘

    │                       │                       │
    v                       v                       v

┌──────────────┐        ┌──────────────┐        ┌──────────────┐
│ Original     │        │ Searchable   │        │ Summary PDF  │
│ Video Stored │        │ Transcript   │        │ with Bluebook │
```

```
│ in Silo                    │      │ with Times                 │      │ Citations        │
└────────────────────────────┘      └────────────────────────────┘      └──────────────────────────┘
```

Supported Media Formats

depotube accepts video in the following formats:

Pre-recorded Upload:

- MP4 (H.264 video codec, AAC audio codec) - recommended
- MOV (QuickTime format with H.264/AAC)
- WebM (VP8 or VP9 video codec)
- Maximum file size: 100GB per deposition
- Minimum resolution: 480p (640×480)
- Recommended resolution: 1080p (1920×1080)

Live Streaming:

- WebRTC protocol only
- Browser requirements: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- Audio/video automatically encoded to H.264/AAC for storage

Unsupported formats: AVI, WMV, FLV, proprietary formats. Files in unsupported formats will be rejected with an error response.

The virtual architecture described above – client silos, user permissions, and deposition processing workflows – is implemented through a **RESTful API** that will be described in detail in **Section 4: Client Management**, **Section 5: Deposition Upload**, **Section 6: Live Deposition Streaming**, and **Section 7: Automatic Transcription**. Each conceptual action (create silo, upload deposition, request summary, etc.) corresponds to specific HTTP endpoints with defined request/response formats.

2.4 Typical Workflows

Typical Workflows

Workflow 0: Admin Setup - Creating a Client Silo

Scenario: A law firm takes on a new case: "Smith, John v. Acme Corporation." The office administrator needs to set up a workspace where attorneys can upload depositions and generate summaries.

Step-by-step:

Step 0: Admin logs in and obtains authentication token

- Admin enters credentials via depotube web interface
- System authenticates and returns an OAuth 2.0 access token
- This token must be included in all subsequent API requests
- Token includes role information (admin, attorney, paralegal)

Step 1: Admin creates the client silo

- Admin calls `POST /api/v1/clients` with client details and authentication token
- System validates admin role from token
- System returns a unique `client_id` (e.g., "client_abc123")
- The silo is now created but empty—no depositions yet

Step 2: Admin grants access to team members

- Admin assigns Attorney Sarah Johnson (attorney role) to this client
- Admin assigns Paralegal Mike Chen (paralegal role) to this client
- System enforces permissions: Sarah can upload/summarize, Mike can only upload

Step 3: Team members can now work

- Sarah logs in (obtains her own authentication token)
- Sarah sees "Smith, John v. Acme Corporation" in her client list
- Mike logs in and sees the same client
- Both can now upload depositions to this silo

API calls involved:

- `POST /api/v1/auth/login` → Authenticate and obtain token
- `POST /api/v1/clients` → Create silo (admin token required)
- `POST /api/v1/clients/{client_id}/permissions` → Grant user access (admin token required)

- `GET /api/v1/clients` → List accessible clients (user sees only their assigned clients)

<u>Note</u>: The following three workflows show the complete depotube experience from setup through value delivery. Note also that all authentications utilize OAuth 2.0, specifically:

*– All authentication uses **OAuth 2.0 access tokens***

*– User roles (admin, attorney, paralegal) are enforced via **OAuth scopes***

*– Login returns an **access token** (and refresh token)*

– All API requests include: `Authorization: Bearer {access_token}`

Workflow 1: Post-Deposition Upload

Scenario: Attorney Sarah Johnson recorded a deposition of the defendant on her laptop. The deposition is 2 hours long and resulted in a 4.5GB MP4 file. She wants to upload it to depotube for transcription and later analysis.

Step-by-step:

Step 0: Sarah logs in

- Sarah opens the depotube web interface and enters her credentials
- System authenticates her and returns an OAuth 2.0 token
- Browser stores this token and includes it in all subsequent API requests
- Sarah's token identifies her as an "attorney" role with access to specific client silos

Step 1: Sarah initiates upload from her browser

- Opens the depotube web interface
- Selects client "Smith, John v. Acme Corporation" (client_abc123)
- Clicks "Upload Deposition" and chooses the MP4 file
- Fills in metadata: deposition date, witness name, case number

- Browser calls `POST /api/v1/clients/client_abc123/depositions/upload`
- System returns a `deposition_id` (e.g., "dep_xyz789")

Step 2: Upload progresses with real-time feedback

- Browser polls `GET /api/v1/depositions/dep_xyz789/upload-progress` every 5 seconds
- Progress bar shows: 0% → 25% → 50% → 75% → 100%
- Upload completes in approximately 8-10 minutes (depending on internet speed)
- User sees: "Upload complete - transcription queued"

Step 2.5: Interrupting upload (abnormal termination)

If Sarah needs to cancel the upload (wrong file, network issues, etc.), there exists a clickable button which does the following:

- Browser calls `DELETE /api/v1/depositions/dep_xyz789`
- Upload is immediately terminated
- Partial file is discarded
- `deposition_id` becomes invalid

No transcription occurs

Step 3: Upload completes normally, and transcription begins automatically

- Once upload reaches 100%, status changes from "uploading" to "transcribing"
- Same polling endpoint now shows transcription progress
- For a 2-hour deposition, transcription takes approximately 30 minutes
- Progress updates: "Transcribing: 15%" → "Transcribing: 45%" → "Transcribing: 85%"
- Status changes to "transcribed" when complete

Step 4: Sarah accesses the transcript

- Sarah calls `GET /api/v1/depositions/dep_xyz789/transcript`
- System returns searchable text with timestamps and speaker identification
- Transcript includes:
    - Speaker labels ("Attorney," "Witness," "Court Reporter")
    - Timestamp for every statement
    - Paragraph breaks and punctuation

- Sarah can now search for specific testimony (e.g., "accident reconstruction")

Step 5: Sarah downloads the transcript (optional)

- Sarah can download as PDF, DOCX, or JSON format
- Downloaded file includes case metadata and certification timestamp

API calls involved:

- `POST /api/v1/auth/login` → Authenticate and obtain token
- `POST /api/v1/clients/{client_id}/depositions/upload` → Initiate upload
- `GET /api/v1/depositions/{deposition_id}/upload-progress` → Monitor progress (polled every 5 seconds)
- `GET /api/v1/depositions/{deposition_id}/transcript` → Retrieve completed transcript
- `GET /api/v1/depositions/{deposition_id}/transcript?format=pdf` → Download as PDF

Timeline:

- Upload (4.5GB file): ~8-10 minutes
- Transcription (2-hour video): ~30 minutes
- **Total time to searchable transcript: ~40 minutes**

Workflow 2: Live Deposition Streaming

Scenario: Attorney Sarah Johnson is conducting a remote deposition via Zoom. She wants to simultaneously stream the deposition to depotube so it's automatically transcribed and archived without requiring a separate upload step after the deposition concludes.

Assuming Sarah is already authenticated from Workflow 1...

Step-by-step:

Step 1: Sarah starts the stream before deposition begins

- Opens depotube web interface in a separate browser tab (Zoom running in another tab)
- Selects client "Smith, John v. Acme Corporation" (client_abc123)
- Clicks "Start Live Stream"
- Enters metadata: witness name, deposition date, case number
- Browser calls `POST /api/v1/clients/client_abc123/depositions/stream/start`
- System returns `deposition_id` and WebRTC connection parameters

Step 2: Browser establishes WebRTC connection

- depotube JavaScript SDK (automatically loaded) handles WebRTC setup
- SDK requests camera/microphone permissions from browser
- SDK establishes peer connection to depotube media servers
- Simple code example:

javascript

```javascript
const stream = await DepotubeStream.start({
  clientId: 'client_abc123',
  metadata: { witness: 'John Defendant', date: '2026-02-03' }
});
// Stream is now live - just 3 lines of code!
```

Step 3: Deposition proceeds with live recording

- Sarah conducts the deposition normally via Zoom
- Her browser simultaneously streams video/audio to depotube via WebRTC
- Transcription engine processes audio in real-time in the background
- Status indicator shows: "Live - Recording" with elapsed time counter
- Deposition lasts 3 hours

Step 3.5 Sarah interrupts the stream (premature termination)

If Sarah needs to cancel the stream (wrong witness, network issues, etc.), there exists a clickable button which terminates the streaming, and does the following:

- Browser calls DELETE /api/v1/depositions/dep_xyz789

- **Stream is immediately terminated**
- Partial recording is discarded
- deposition_id becomes invalid

  No transcription occurs

Step 4: Sarah ends the stream (normal termination)

- Deposition concludes
- Sarah clicks "End Stream" button in depotube interface
- Browser calls `POST /api/v1/depositions/{deposition_id}/stream/stop`
- WebRTC connection closes gracefully
- Browser code:

javascript

```
await stream.stop();

// Stream ended and saved
```

Step 5: Transcript becomes available immediately

- Since transcription happened during streaming, transcript is ready when stream stops
- No additional processing time required
- Sarah can immediately access via `GET /api/v1/depositions/{deposition_id}/transcript`
- Transcript includes same features: timestamps, speaker identification, searchability

Step 6: Video archive is ready

- Recorded stream is saved as MP4 file
- Available for download via `GET /api/v1/depositions/{deposition_id}/video`
- File size: ~3.2GB for 3-hour stream at 720p

API calls involved:

- `POST /api/v1/clients/{client_id}/depositions/stream/start` → Begin WebRTC stream
- depotube JavaScript SDK → Handles all WebRTC complexity
- `POST /api/v1/depositions/{deposition_id}/stream/stop` → End stream
- `GET /api/v1/depositions/{deposition_id}/transcript` → Retrieve transcript (available immediately)
- `GET /api/v1/depositions/{deposition_id}/video` → Download archived video

Timeline:

- Stream duration: 3 hours (live)
- Transcription processing: 0 minutes (happened during stream)
- **Total time to searchable transcript: Immediate upon stream end**

Key advantage: No waiting period for transcription, unlike post-deposition upload.

Workflow 3: AI Summary Generation

Scenario: After reviewing a deposition transcript, Attorney Sarah Johnson wants an AI-generated summary that identifies key testimony and cites relevant case law. This will help her prepare cross-examination questions for trial and identify weaknesses in the witness's testimony for her motion for summary judgment.

Assuming Sarah is already authenticated and has a completed transcript from Workflow 1 or 2...

Step-by-step:

Step 1: Sarah requests AI summary

- Opens the deposition detail page in depotube interface
- Reviews the transcript and decides she needs case law analysis
- Clicks "Generate AI Summary" button
- Browser calls `POST /api/v1/depositions/dep_xyz789/summarize`
- System queues the analysis job and returns:

json

```
{
  "summary_job_id": "sum_job_456",
  "status": "processing",
  "estimated_completion": "2026-02-03T15:45:00Z"
}
```

- Estimated time: 10 minutes for a 2-hour deposition

Step 2: Summary generation progresses

- Browser polls `GET /api/v1/depositions/dep_xyz789/summary` every 10 seconds
- Status updates:
  - "Analyzing transcript..." (0-3 minutes)
  - "Searching case law database..." (3-7 minutes)
  - "Generating citations..." (7-10 minutes)
  - "Complete" (10 minutes)

Step 3: Sarah retrieves the completed summary

- Summary is now available
- Browser calls `GET /api/v1/depositions/dep_xyz789/summary`
- System returns structured JSON with:

Executive Overview:

> "The defendant testified under oath that he was not present at the accident scene on January 15, 2026. However, testimony contradicts cell phone location data presented during discovery. Key admissions include acknowledgment of prior vehicle violations and uncertainty about vehicle maintenance records."

Key Facts Admitted:

- Defendant confirmed ownership of 2018 Honda Accord (VIN: 1HGBH...)
- Admitted to three prior speeding violations in 2024-2025
- Could not recall last brake inspection date
- Acknowledged receiving traffic citation on January 10, 2026

Contradictions and Weaknesses:

- Claims he was "at home" on January 15 at 3:00 PM (transcript p.45, ln.12-15)
- Cell phone records show device location within 500 feet of accident scene at 3:07 PM
- Used hedging language 47 times ("I think," "maybe," "I'm not sure")
- Testimony conflicts with earlier written declaration (filed March 2025)

Case Law Citations:

1. *Daubert v. Merrell Dow Pharmaceuticals, Inc.*, 509 U.S. 579 (1993)
    - Relevance: Expert testimony regarding cell phone location data
    - Holding: Scientific evidence must meet reliability standards
    - Application: Cell phone geolocation data widely accepted as reliable evidence
    - Transcript reference: p.67, ln.8-24 (defendant challenges location data validity)
2. *Anderson v. Liberty Lobby, Inc.*, 477 U.S. 242 (1986)
    - Relevance: Summary judgment standard when credibility is disputed
    - Holding: Contradictory testimony creates genuine issue of material fact
    - Application: Defendant's conflicting statements may preclude summary judgment
    - Transcript reference: p.45, ln.12-15 vs. p.89, ln.3-7
3. *Celotex Corp. v. Catrett*, 477 U.S. 317 (1987)
    - Relevance: Burden of proof on summary judgment motion
    - Application: Plaintiff has raised sufficient doubt to survive summary judgment

Step 4: Sarah reviews and exports the summary

- Reviews AI-generated analysis for accuracy
- Notes that case law citations are properly Bluebook formatted
- Clicks "Export as PDF"
- Downloads formatted summary document with:
    - Law firm letterhead
    - Case caption
    - Table of contents
    - Footnoted case citations
    - Cross-references to transcript line numbers

Step 5: Sarah uses summary in legal work

- Incorporates case law citations into her motion for summary judgment
- Uses "Contradictions and Weaknesses" section to prepare cross-examination questions
- Shares PDF summary with co-counsel for trial preparation
- Saves 6-8 hours of manual legal research and analysis

API calls involved:

- `POST /api/v1/depositions/{deposition_id}/summarize` → Request AI analysis
- `GET /api/v1/depositions/{deposition_id}/summary` → Check status / retrieve summary (polled every 10 seconds)
- `GET /api/v1/depositions/{deposition_id}/summary?format=pdf` → Download formatted PDF

Timeline:

- AI analysis and case law search: ~10 minutes
- Attorney review and refinement: ~30-45 minutes
- **Total time: ~1 hour** (vs. 6-9 hours manually)

Cost savings:

- Attorney billing rate: $350/hour
- Traditional research time: 8 hours = $2,800
- With depotube: 1 hour = $350
- **Savings: $2,450 per deposition**

2.5 Document scope and Forward Compatibility

What This Document Covers:

This functional specification describes the **RESTful API interface** for depotube - the contract between client applications (web browsers, mobile apps, third-party integrations) and the depotube platform. It defines:

- HTTP endpoints (URLs, methods, parameters)
- Request formats (JSON structure, multipart uploads, authentication headers)
- Response formats (JSON structure, status codes, error messages)

- Client-side integration patterns (JavaScript SDK usage, polling strategies, WebRTC setup)
- Expected behavior and workflows from the client perspective

What This Document Does NOT Cover:

This document intentionally does **not** describe backend implementation details, including:

- **Database schema** - How clients, depositions, and transcripts are stored internally
- **Transcription engine** - Speech-to-text algorithms, model architecture, or training data
- **AI summary generation** - Machine learning models, case law database structure, or citation extraction logic
- **Server infrastructure** - Cloud architecture, load balancing, storage systems, or scaling strategies
- **Video processing pipeline** - Encoding, compression, or streaming server implementation
- **Security implementation** - OAuth provider configuration, encryption methods, or key management
- **Frontend web application** - UI/UX design, HTML/CSS styling, page layouts, navigation flows, or client-side application logic (beyond the API integration examples provided)

Why this separation matters:

Backend implementation may change (e.g., switching transcription providers, upgrading AI models, migrating databases) **without affecting the API contract**. As long as the RESTful API endpoints maintain their defined behavior, client applications continue to work unchanged.

API Stability and Forward Compatibility

depotube's commitment:

The RESTful API interfaces and architecture described in this document represent a **stable contract** between depotube and client applications. depotube commits to maintaining these interfaces for the foreseeable future, regardless of backend implementation changes.

What this means in practice:

- Endpoint URLs remain stable
- Request/response formats are maintained
- Authentication mechanisms are preserved
- Behavioral contracts are honored

Backend changes that do NOT affect the API:

- Upgrading transcription engine providers
- Migrating databases
- Switching cloud infrastructure
- Improving AI algorithms

When breaking changes are necessary: If depotube must introduce breaking changes (rare), we will:

1. Version the API (e.g., `/api/v2/...`)
2. Provide 12-month migration period
3. Document migration path
4. Notify developers

3. Authentication

3.1 OAuth 2.0 Overview

depotube uses **OAuth 2.0** for authentication and authorization. All API requests must include a valid access token in the Authorization header.

OAuth 2.0 provides:

- **Secure authentication** without exposing user passwords to client applications
- **Scoped permissions** that limit what actions a token can perform
- **Token expiration** and refresh mechanisms for long-lived sessions
- **Third-party integration** support for external applications

3.2 OAuth 2.0 Authorization Flow

Step 1: Initiate Authorization

Direct the user's browser to the authorization endpoint:

```
GET https://api.depotube.com/oauth/authorize
  ?response_type=code
  &client_id=YOUR_CLIENT_ID
  &redirect_uri=https://yourapp.com/callback
  &scope=attorney client:read deposition:upload deposition:summarize
  &state=random_state_string
```

Parameters:

- `response_type`: Must be `code` for authorization code flow
- `client_id`: Your application's client ID (obtained during registration)
- `redirect_uri`: Where to redirect after authorization (must match registered URI)
- `scope`: Space-separated list of requested permissions (see Section 3.3)
- `state`: Random string to prevent CSRF attacks (recommended)

Step 2: User Authorizes Application

The user sees a depotube login page, enters credentials, and approves the requested scopes.

Step 3: Receive Authorization Code

depotube redirects back to your `redirect_uri` with an authorization code:

```
https://yourapp.com/callback
  ?code=AUTH_CODE_HERE
  &state=random_state_string
```

Step 4: Exchange Code for Access Token

Your server makes a POST request to exchange the code for an access token:

http

POST https://api.depotube.com/oauth/token

Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code

&code=AUTH_CODE_HERE

&client_id=YOUR_CLIENT_ID

&client_secret=YOUR_CLIENT_SECRET

&redirect_uri=https://yourapp.com/callback

Response:


```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "tGzv3JOkF0XG5Qx2TlKWIA",
  "scope": "attorney client:read deposition:upload deposition:summarize"
}
```


Step 5: Use Access Token in API Requests

Include the access token in all API requests:

http

GET https://api.depotube.com/v1/clients

Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...

Step 6: Refresh Expired Tokens

When the access token expires (after 3600 seconds), use the refresh token:

http

POST https://api.depotube.com/oauth/token

Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token

&refresh_token=tGzv3JOkF0XG5Qx2TlKWIA

&client_id=YOUR_CLIENT_ID

&client_secret=YOUR_CLIENT_SECRET

Response:

json

```
{
  "access_token": "NEW_ACCESS_TOKEN",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "NEW_REFRESH_TOKEN",
  "scope": "attorney client:read deposition:upload deposition:summarize"
}
```

3.3 OAuth Scopes and Permissions

Scopes define what actions a token can perform. Each user role has associated scopes:

Admin Scopes

Scope	Description
`admin`	Full administrative access (includes all scopes below)
`client:create`	Create new client silos
`client:read`	View client information
`client:update`	Modify client metadata
`client:delete`	Delete client silos
`permission:manage`	Grant/revoke user access to client silos
`deposition:upload`	Upload pre-recorded depositions
`deposition:stream`	Stream live depositions
`deposition:read`	View deposition metadata and status

`deposition:download`	Download videos, transcripts, summaries
`deposition:summarize`	Request AI summary generation
`deposition:delete`	Delete depositions
`audit:read`	View audit logs

Attorney Scopes

Scope	Description
`attorney`	Attorney role (includes scopes below)
`client:read`	View client information (for assigned clients only)
`deposition:upload`	Upload pre-recorded depositions
`deposition:stream`	Stream live depositions

`deposition:read`	View deposition metadata and status
`deposition:download`	Download videos, transcripts, summaries
`deposition:summarize`	Request AI summary generation
`audit:read`	View audit logs (for assigned clients only)

Paralegal Scopes

Scope	Description
`paralegal`	Paralegal role (includes scopes below)
`client:read`	View client information (for assigned clients only)
`deposition:upload`	Upload pre-recorded depositions
`deposition:read`	View deposition metadata and status

`deposition:download`	Download transcripts and summaries only (not video)
`audit:read`	View limited audit log information

3.4 Token Security

Best Practices:

- **Never expose tokens in client-side code** - Store access tokens securely on your server
- **Use HTTPS only** - All API requests must use TLS/SSL encryption
- **Rotate refresh tokens** - Use the new refresh token returned with each token refresh
- **Revoke compromised tokens** - Call `/oauth/revoke` if a token is compromised
- **Set appropriate scopes** - Request only the minimum scopes needed for your application

Token Revocation:

http

POST https://api.depotube.com/oauth/revoke

Content-Type: application/x-www-form-urlencoded

Authorization: Bearer YOUR_ACCESS_TOKEN

token=YOUR_ACCESS_TOKEN

&token_type_hint=access_token

3.5 Error Responses

Invalid or expired token:

http

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json

{
  "error": "invalid_token",
  "error_description": "The access token expired"
}
```

Insufficient permissions:

http

```
HTTP/1.1 403 Forbidden
Content-Type: application/json

{
  "error": "insufficient_scope",
  "error_description": "Token does not have 'client:delete' scope"
}
```

---

4. Client Management

4.1 Overview

A **client** represents a person or entity involved in litigation. Each client has an isolated silo containing all associated depositions, transcripts, and summaries. Only users with explicit access to a client can view or modify its data.

4.2 Create Client

Endpoint: `POST /api/v1/clients`

Required Scope: `admin` or `client:create`

Description: Create a new client silo. Only administrators can create clients.

Request:

http

POST https://api.depotube.com/v1/clients

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: application/json

```
{
  "name": "Smith, John",
  "case_number": "CV-2026-12345",
  "matter_description": "Personal injury claim - vehicular accident",
  "attorney_of_record": "Sarah Johnson",
  "notes": "Defendant in multi-party litigation"
}
```

Request Parameters:

Field	Type	Required	Description
`name`	string	Yes	Client name (person or entity)
`case_number`	string	No	Court case number or internal matter ID
`matter_description`	string	No	Brief description of the legal matter
`attorney_of_record`	string	No	Primary attorney handling the case
`notes`	string	No	Additional notes or context

Response:

http

HTTP/1.1 201 Created

Content-Type: application/json


```
{
  "client_id": "client_abc123",
  "name": "Smith, John",
```

```
  "case_number": "CV-2026-12345",
  "matter_description": "Personal injury claim - vehicular accident",
  "attorney_of_record": "Sarah Johnson",
  "notes": "Defendant in multi-party litigation",
  "created_at": "2026-02-03T14:30:00Z",
  "updated_at": "2026-02-03T14:30:00Z",
  "deposition_count": 0
}
```


Response Fields:

Field	Type	Description
`client_id`	string	Unique identifier for this client
`name`	string	Client name
`case_number`	string	Court case number
`matter_description`	string	Legal matter description
`attorney_of_record`	string	Primary attorney

`notes`	string	Additional notes
`created_at`	ISO 8601 datetime	When the client was created
`updated_at`	ISO 8601 datetime	When the client was last modified
`deposition_count`	integer	Number of depositions in this silo

4.3 List Clients

Endpoint: `GET /api/v1/clients`

Required Scope: `client:read`

Description: Retrieve all clients accessible to the authenticated user. Users only see clients they have been granted access to.

Request:

http

GET https://api.depotube.com/v1/clients

Authorization: Bearer YOUR_ACCESS_TOKEN

Optional Query Parameters:

Parameter	Type	Description

`limit`	integer	Number of results per page (default: 50, max: 100)
`offset`	integer	Number of results to skip for pagination (default: 0)
`sort`	string	Sort field: `name`, `created_at`, `updated_at` (default: `name`)
`order`	string	Sort order: `asc` or `desc` (default: `asc`)

Example with Pagination:

http

GET https://api.depotube.com/v1/clients?limit=20&offset=40&sort=created_at&order=desc

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "clients": [
    {
```

```
    "client_id": "client_abc123",
    "name": "Smith, John",
    "case_number": "CV-2026-12345",
    "matter_description": "Personal injury claim - vehicular accident",
    "attorney_of_record": "Sarah Johnson",
    "created_at": "2026-02-03T14:30:00Z",
    "updated_at": "2026-02-03T14:30:00Z",
    "deposition_count": 3
  },
  {
    "client_id": "client_def456",
    "name": "Acme Corporation",
    "case_number": "CV-2025-98765",
    "matter_description": "Contract dispute",
    "attorney_of_record": "Michael Chen",
    "created_at": "2025-11-15T09:20:00Z",
    "updated_at": "2026-01-20T16:45:00Z",
    "deposition_count": 7
  }
],
"total": 2,
"limit": 50,
"offset": 0
```

}

Response Fields:

Field	Type	Description
`clients`	array	Array of client objects
`total`	integer	Total number of accessible clients
`limit`	integer	Results per page
`offset`	integer	Current pagination offset

4.4 Get Client

Endpoint: `GET /api/v1/clients/{client_id}`

Required Scope: `client:read`

Description: Retrieve detailed information about a specific client.

Request:

http

GET https://api.depotube.com/v1/clients/client_abc123

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "client_id": "client_abc123",
  "name": "Smith, John",
  "case_number": "CV-2026-12345",
  "matter_description": "Personal injury claim - vehicular accident",
  "attorney_of_record": "Sarah Johnson",
  "notes": "Defendant in multi-party litigation",
  "created_at": "2026-02-03T14:30:00Z",
  "updated_at": "2026-02-03T14:30:00Z",
  "deposition_count": 3,
  "total_storage_bytes": 15000000000,
  "recent_depositions": [
    {
      "deposition_id": "dep_xyz789",
      "witness_name": "John Defendant",
      "deposition_date": "2026-02-01",
      "status": "transcribed",
      "created_at": "2026-02-01T10:15:00Z"
```

```
    },
    {
      "deposition_id": "dep_abc456",
      "witness_name": "Jane Witness",
      "deposition_date": "2026-01-25",
      "status": "summarized",
      "created_at": "2026-01-25T14:30:00Z"
    }
  ]
}
```


4.5 Update Client

Endpoint: `PATCH /api/v1/clients/{client_id}`

Required Scope: `admin` or `client:update`

Description: Update client metadata. Only provided fields are updated.

Request:

http

PATCH https://api.depotube.com/v1/clients/client_abc123

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: application/json


```
{
  "attorney_of_record": "Michael Chen",
```

```
  "notes": "Defendant in multi-party litigation. Case settled on 2026-02-15."
}
```

Response:

http

```
HTTP/1.1 200 OK
Content-Type: application/json


{
  "client_id": "client_abc123",
  "name": "Smith, John",
  "case_number": "CV-2026-12345",
  "matter_description": "Personal injury claim - vehicular accident",
  "attorney_of_record": "Michael Chen",
  "notes": "Defendant in multi-party litigation. Case settled on 2026-02-15.",
  "created_at": "2026-02-03T14:30:00Z",
  "updated_at": "2026-02-15T11:20:00Z",
  "deposition_count": 3
}
```

4.6 Delete Client

Endpoint: `DELETE /api/v1/clients/{client_id}`

Required Scope: `admin` or `client:delete`

Description: Permanently delete a client silo and all associated depositions, transcripts, and summaries. This operation cannot be undone. An audit log entry is created for compliance purposes.

Request:

http

DELETE https://api.depotube.com/v1/clients/client_abc123

Authorization: Bearer YOUR_ACCESS_TOKEN

Optional Query Parameter:

Parameter	Type	Description
`confirm`	boolean	Must be `true` to actually delete (safety check)

Example:

http

DELETE https://api.depotube.com/v1/clients/client_abc123?confirm=true

Authorization: Bearer YOUR_ACCESS_TOKEN

Response (successful deletion):

http

HTTP/1.1 200 OK

Content-Type: application/json

{

```
  "client_id": "client_abc123",
  "deleted": true,
  "depositions_deleted": 3,
  "total_bytes_freed": 15000000000,
  "deleted_at": "2026-02-03T15:45:00Z",
  "audit_log_id": "audit_log_12345"
}
```


Response (missing confirmation):

http

HTTP/1.1 400 Bad Request

Content-Type: application/json


```
{
  "error": "confirmation_required",
  "error_description": "Add '?confirm=true' to delete this client and all associated depositions"
}
```


4.7 Manage User Permissions

Endpoint: `POST /api/v1/clients/{client_id}/permissions`

Required Scope: `admin` or `permission:manage`

Description: Grant or revoke user access to a specific client silo.

Request (grant access):

http

POST https://api.depotube.com/v1/clients/client_abc123/permissions

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: application/json

```
{
  "user_email": "sarah.johnson@lawfirm.com",
  "role": "attorney",
  "action": "grant"
}
```

Request Parameters:

Field	Type	Required	Description
`user_email`	string	Yes	Email address of the user
`role`	string	Yes	Role to assign: `admin`, `attorney`, or `paralegal`
`action`	string	Yes	Action to perform: `grant` or `revoke`

Response:

http

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "client_id": "client_abc123",
  "user_email": "sarah.johnson@lawfirm.com",
  "role": "attorney",
  "granted_at": "2026-02-03T14:35:00Z",
  "granted_by": "admin@lawfirm.com"
}
```

Request (revoke access):

http

```
POST https://api.depotube.com/v1/clients/client_abc123/permissions
Authorization: Bearer YOUR_ACCESS_TOKEN
Content-Type: application/json

{
  "user_email": "sarah.johnson@lawfirm.com",
  "action": "revoke"
}
```

Response:

http

```
HTTP/1.1 200 OK

Content-Type: application/json


{

  "client_id": "client_abc123",

  "user_email": "sarah.johnson@lawfirm.com",

  "revoked": true,

  "revoked_at": "2026-02-03T15:10:00Z",

  "revoked_by": "admin@lawfirm.com"

}
```

---

5. Deposition Upload

5.1 Overview

Upload pre-recorded deposition videos from a web browser or server application. The upload process includes:

1. Initiate multipart upload with deposition metadata
2. Transfer video file via HTTP POST
3. Poll for upload progress
4. Automatic transcription begins when upload completes

5.2 Initiate Upload

Endpoint: `POST /api/v1/clients/{client_id}/depositions/upload`

Required Scope: `attorney` or `deposition:upload`

Description: Upload a pre-recorded deposition video file and trigger automatic transcription.

Request:

http

POST https://api.depotube.com/v1/clients/client_abc123/depositions/upload

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

------WebKitFormBoundary7MA4YWxkTrZu0gW

Content-Disposition: form-data; name="video"; filename="deposition_2026-02-01.mp4"

Content-Type: video/mp4

[binary video data]

------WebKitFormBoundary7MA4YWxkTrZu0gW

Content-Disposition: form-data; name="metadata"

Content-Type: application/json


```
{
  "witness_name": "John Defendant",
  "deposition_date": "2026-02-01",
  "attorney_names": ["Sarah Johnson", "Michael Chen"],
  "case_number": "CV-2026-12345",
  "location": "San Francisco, CA",
```

"duration_seconds": 7200,

"notes": "Defendant testimony regarding accident on January 15, 2026"

}

------WebKitFormBoundary7MA4YWxkTrZu0gW--

Metadata Parameters:

Field	Type	Required	Description
`witness_name`	string	Yes	Name of person being deposed
`deposition_date`	ISO 8601 date	Yes	Date deposition was recorded
`attorney_names`	array of strings	No	Attorneys present at deposition
`case_number`	string	No	Court case number
`location`	string	No	Where deposition took place
`duration_seconds`	integer	No	Video length in seconds (optional, auto-detected)
`notes`	string	No	Additional context or notes

Response (upload initiated):

http

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "deposition_id": "dep_xyz789",
  "client_id": "client_abc123",
  "witness_name": "John Defendant",
  "deposition_date": "2026-02-01",
  "status": "uploading",
  "upload_percent": 0,
  "bytes_uploaded": 0,
  "total_bytes": 4500000000,
  "created_at": "2026-02-01T10:15:00Z",
  "estimated_upload_completion": "2026-02-01T10:25:00Z"
}
```

5.3 JavaScript Upload Example

Browser-based upload with progress tracking:

javascript

```
// HTML file input
const fileInput = document.getElementById('videoFile');
```

```
const videoFile = fileInput.files[0];

// Prepare metadata
const metadata = {
  witness_name: 'John Defendant',
  deposition_date: '2026-02-01',
  attorney_names: ['Sarah Johnson', 'Michael Chen'],
  case_number: 'CV-2026-12345'
};

// Create FormData
const formData = new FormData();
formData.append('video', videoFile);
formData.append('metadata', JSON.stringify(metadata));

// Upload with fetch
const response = await fetch(
  'https://api.depotube.com/v1/clients/client_abc123/depositions/upload',
  {
    method: 'POST',
    headers: {
      'Authorization': 'Bearer YOUR_ACCESS_TOKEN'
      // Don't set Content-Type - browser sets it with boundary
```

```
    },
    body: formData
  }
);

const result = await response.json();
const depositionId = result.deposition_id;

// Start polling for progress
pollUploadProgress(depositionId);
```

5.4 Poll Upload Progress

Endpoint: `GET /api/v1/depositions/{deposition_id}/upload-progress`

Required Scope: `deposition:read`

Description: Check the status of an in-progress upload. Poll this endpoint every 5 seconds during upload.

Request:

http

```
GET https://api.depotube.com/v1/depositions/dep_xyz789/upload-progress
Authorization: Bearer YOUR_ACCESS_TOKEN
```

Response (upload in progress):

http

```
HTTP/1.1 200 OK
```

Content-Type: application/json


```
{
  "deposition_id": "dep_xyz789",
  "status": "uploading",
  "upload_percent": 67,
  "bytes_uploaded": 3015000000,
  "total_bytes": 4500000000,
  "upload_speed_mbps": 45.2,
  "estimated_completion": "2026-02-01T10:22:00Z"
}
```


Response (upload complete, transcription queued):

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "deposition_id": "dep_xyz789",
  "status": "upload_complete",
  "upload_percent": 100,
  "bytes_uploaded": 4500000000,
  "total_bytes": 4500000000,
  "completed_at": "2026-02-01T10:23:00Z",
```

```
  "transcription_status": "queued",
  "next_step": "transcription"
}
```

Response (transcription in progress):

http

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "deposition_id": "dep_xyz789",
  "status": "transcribing",
  "upload_percent": 100,
  "transcription_percent": 34,
  "estimated_transcription_completion": "2026-02-01T10:50:00Z"
}
```

Response (transcription complete):

http

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "deposition_id": "dep_xyz789",
```

```
  "status": "transcribed",
  "upload_percent": 100,
  "transcription_percent": 100,
  "transcription_completed_at": "2026-02-01T10:48:00Z",
  "transcript_available": true,
  "transcript_url": "https://api.depotube.com/v1/depositions/dep_xyz789/transcript"
}
```

5.5 JavaScript Polling Example

javascript

```javascript
async function pollUploadProgress(depositionId) {
  const pollInterval = setInterval(async () => {
    try {
      const response = await fetch(
        `https://api.depotube.com/v1/depositions/${depositionId}/upload-progress`,
        {
          headers: {
            'Authorization': 'Bearer YOUR_ACCESS_TOKEN'
          }
        }
      );

      const progress = await response.json();
```

```
    // Update UI
    if (progress.status === 'uploading') {
      console.log(`Upload: ${progress.upload_percent}%`);
      updateProgressBar(progress.upload_percent);
    } else if (progress.status === 'transcribing') {
      console.log(`Transcribing: ${progress.transcription_percent}%`);
      updateProgressBar(100 + progress.transcription_percent); // Combined progress
    } else if (progress.status === 'transcribed') {
      console.log('Transcription complete!');
      clearInterval(pollInterval);
      showTranscript(depositionId);
    }

  } catch (error) {
    console.error('Polling error:', error);
    clearInterval(pollInterval);
  }
}, 5000); // Poll every 5 seconds
}
```

5.6 Cancel Upload

Endpoint: `DELETE /api/v1/depositions/{deposition_id}`

Required Scope: `admin` or `deposition:delete`

Description: Cancel an in-progress upload. Partial data is discarded.

Request:

http

DELETE https://api.depotube.com/v1/depositions/dep_xyz789

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "deposition_id": "dep_xyz789",
  "status": "cancelled",
  "upload_percent": 45,
  "cancelled_at": "2026-02-01T10:20:00Z",
  "bytes_discarded": 2025000000
}
```

5.7 Supported Video Formats

Format	Container	Video Codec	Audio Codec	Max File Size

MP4	MPEG-4	H.264 (AVC)	AAC	100 GB
MOV	QuickTime	H.264 (AVC)	AAC	100 GB
WebM	WebM	VP8 or VP9	Vorbis or Opus	100 GB

Rejected formats: AVI, WMV, FLV, MKV, or proprietary codecs

Resolution requirements:

- Minimum: 480p (640×480)
- Recommended: 1080p (1920×1080)
- Maximum: 4K (3840×2160)

Frame rate: 24-60 fps (30 fps recommended)

Bitrate: 1-50 Mbps (5 Mbps recommended for 1080p)

---

6. Live Deposition Streaming

6.1 Overview

Stream live depositions directly from a web browser using WebRTC. The depotube JavaScript SDK handles all WebRTC complexity, providing a simple API for starting and stopping streams.

Key features:

- Real-time video/audio capture from browser
- Automatic transcription during streaming

- No media server infrastructure required on client side
- Immediate transcript availability when stream ends

6.2 Prerequisites

Supported Browsers:

- Chrome 90+
- Firefox 88+
- Safari 14+
- Edge 90+

Browser Permissions:

- Camera access
- Microphone access

Network Requirements:

- Minimum 2 Mbps upload speed
- Recommended 5 Mbps upload speed for 720p
- Open UDP ports for WebRTC (STUN/TURN)

6.3 Include JavaScript SDK

Add the depotube SDK to your HTML page:

html

```html
<script src="https://cdn.depotube.com/sdk/depotube-stream-v1.min.js"></script>
```

Or install via npm:

bash

```bash
npm install @depotube/stream-sdk
```

Then import:

javascript

```
import { DepotubeStream } from '@depotube/stream-sdk';
```

6.4 Start Live Stream

Endpoint: `POST /api/v1/clients/{client_id}/depositions/stream/start`

Required Scope: `attorney` or `deposition:stream`

Description: Initiate a live deposition stream. Returns WebRTC connection parameters.

Request:

```
http
POST https://api.depotube.com/v1/clients/client_abc123/depositions/stream/start
Authorization: Bearer YOUR_ACCESS_TOKEN
Content-Type: application/json

{
  "witness_name": "Jane Witness",
  "deposition_date": "2026-02-03",
  "attorney_names": ["Sarah Johnson"],
  "case_number": "CV-2026-12345",
  "location": "Remote (Zoom)",
  "video_quality": "720p"
}
```

Request Parameters:

Field	Type	Required	Description
`witness_name`	string	Yes	Name of person being deposed
`deposition_date`	ISO 8601 date	Yes	Date of deposition
`attorney_names`	array of strings	No	Attorneys present
`case_number`	string	No	Court case number
`location`	string	No	Deposition location
`video_quality`	string	No	Quality: `480p`, `720p`, `1080p` (default: `720p`)

Response:

http

HTTP/1.1 201 Created

Content-Type: application/json


```
{
  "deposition_id": "dep_live_123",
```

```
  "client_id": "client_abc123",
  "witness_name": "Jane Witness",
  "status": "waiting_for_stream",
  "webrtc_config": {
    "ice_servers": [
      {
        "urls": "stun:stun.depotube.com:3478"
      },
      {
        "urls": "turn:turn.depotube.com:3478",
        "username": "user_xyz",
        "credential": "temp_credential_abc"
      }
    ],
    "media_server_url": "wss://media.depotube.com/ingest/dep_live_123",
    "session_token": "sess_token_789"
  },
  "created_at": "2026-02-03T14:00:00Z"
}
```


6.5 JavaScript SDK Usage

Simple streaming with SDK (recommended):

javascript

```
// Initialize SDK with auth token
DepotubeStream.configure({
  apiToken: 'YOUR_ACCESS_TOKEN',
  apiBaseUrl: 'https://api.depotube.com/v1'
});

// Start stream
const stream = await DepotubeStream.start({
  clientId: 'client_abc123',
  metadata: {
    witness_name: 'Jane Witness',
    deposition_date: '2026-02-03',
    attorney_names: ['Sarah Johnson'],
    case_number: 'CV-2026-12345',
    location: 'Remote (Zoom)',
    video_quality: '720p'
  },
  onStatusChange: (status) => {
    console.log('Stream status:', status);
    // Status values: 'initializing', 'connecting', 'live', 'stopped', 'error'
  },
  onError: (error) => {
    console.error('Stream error:', error);
```

```
  }
});

// Stream is now live - deposition_id available
console.log('Deposition ID:', stream.depositionId);

// Display local video preview (optional)
const videoElement = document.getElementById('preview');
videoElement.srcObject = stream.getLocalStream();

// Stop stream when deposition ends
await stream.stop();
console.log('Stream stopped, transcript available immediately');
```

Complete HTML Example:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>depotube Live Stream</title>
  <script src="https://cdn.depotube.com/sdk/depotube-stream-v1.min.js"></script>
</head>
<body>
  <h1>Live Deposition Stream</h1>
```

```html
<div>
  <video id="preview" autoplay muted width="640" height="480"></video>
</div>

<div>
  <button id="startBtn">Start Stream</button>
  <button id="stopBtn" disabled>Stop Stream</button>
  <p id="status">Not streaming</p>
</div>

<script>
  let activeStream = null;

  // Configure SDK
  DepotubeStream.configure({
    apiToken: 'YOUR_ACCESS_TOKEN',
    apiBaseUrl: 'https://api.depotube.com/v1'
  });

  // Start button
  document.getElementById('startBtn').addEventListener('click', async () => {
    try {
```

```
    activeStream = await DepotubeStream.start({
      clientId: 'client_abc123',
      metadata: {
        witness_name: 'Jane Witness',
        deposition_date: '2026-02-03',
        attorney_names: ['Sarah Johnson']
      },
      onStatusChange: (status) => {
        document.getElementById('status').textContent = `Status: ${status}`;
        if (status === 'live') {
          document.getElementById('startBtn').disabled = true;
          document.getElementById('stopBtn').disabled = false;
        }
      }
    });

    // Show preview
    document.getElementById('preview').srcObject = activeStream.getLocalStream();

  } catch (error) {
    alert('Failed to start stream: ' + error.message);
  }
});
```

```
  // Stop button
  document.getElementById('stopBtn').addEventListener('click', async () => {
    if (activeStream) {
      await activeStream.stop();
      document.getElementById('startBtn').disabled = false;
      document.getElementById('stopBtn').disabled = true;
      document.getElementById('status').textContent = 'Stream stopped';
      document.getElementById('preview').srcObject = null;
    }
  });
 </script>
</body>
</html>
```

6.6 Manual WebRTC Implementation (Advanced)

If you prefer not to use the SDK, you can implement WebRTC manually:

javascript

```
// 1. Call API to start stream
const response = await fetch(
  'https://api.depotube.com/v1/clients/client_abc123/depositions/stream/start',
  {
    method: 'POST',
```

```
    headers: {
      'Authorization': 'Bearer YOUR_ACCESS_TOKEN',
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      witness_name: 'Jane Witness',
      deposition_date: '2026-02-03'
    })
  }
);

const streamData = await response.json();
const depositionId = streamData.deposition_id;
const webrtcConfig = streamData.webrtc_config;

// 2. Create peer connection
const pc = new RTCPeerConnection({
  iceServers: webrtcConfig.ice_servers
});

// 3. Get user media
const localStream = await navigator.mediaDevices.getUserMedia({
  video: { width: 1280, height: 720 },
```

```
  audio: true
});

// 4. Add tracks to peer connection
localStream.getTracks().forEach(track => {
  pc.addTrack(track, localStream);
});

// 5. Create offer
const offer = await pc.createOffer();
await pc.setLocalDescription(offer);

// 6. Send offer to media server via WebSocket
const ws = new WebSocket(webrtcConfig.media_server_url);
ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'offer',
    sdp: offer.sdp,
    session_token: webrtcConfig.session_token
  }));
};

ws.onmessage = async (event) => {
```

```
  const message = JSON.parse(event.data);
  if (message.type === 'answer') {
    await pc.setRemoteDescription({
      type: 'answer',
      sdp: message.sdp
    });
  } else if (message.type === 'ice_candidate') {
    await pc.addIceCandidate(message.candidate);
  }
};

// 7. Handle ICE candidates
pc.onicecandidate = (event) => {
  if (event.candidate) {
    ws.send(JSON.stringify({
      type: 'ice_candidate',
      candidate: event.candidate
    }));
  }
};

// Stream is now live!
```

6.7 Stop Live Stream

Endpoint: `POST /api/v1/depositions/{deposition_id}/stream/stop`

Required Scope: `attorney` or `deposition:stream`

Description: End a live stream. Transcript becomes available immediately.

Request:

http

POST https://api.depotube.com/v1/depositions/dep_live_123/stream/stop

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "deposition_id": "dep_live_123",
  "status": "stream_ended",
  "stream_duration_seconds": 10800,
  "stopped_at": "2026-02-03T17:00:00Z",
  "video_size_bytes": 3200000000,
  "transcript_available": true,
  "transcript_url": "https://api.depotube.com/v1/depositions/dep_live_123/transcript"
}
```

6.8 Monitor Stream Status

Endpoint: `GET /api/v1/depositions/{deposition_id}/stream-status`

Required Scope: `deposition:read`

Description: Check the status of an active stream.

Request:

http

```
GET https://api.depotube.com/v1/depositions/dep_live_123/stream-status

Authorization: Bearer YOUR_ACCESS_TOKEN
```

Response (stream active):

http

```
HTTP/1.1 200 OK

Content-Type: application/json


{

  "deposition_id": "dep_live_123",

  "status": "streaming",

  "started_at": "2026-02-03T14:00:00Z",

  "elapsed_seconds": 3600,

  "current_bitrate_kbps": 2500,

  "frames_received": 108000,

  "connection_quality": "excellent"

}
```

6.9 Error Handling

Camera/microphone permission denied:

javascript

```javascript
try {
  const stream = await DepotubeStream.start({...});
} catch (error) {
  if (error.name === 'NotAllowedError') {
    alert('Please grant camera and microphone permissions to stream depositions.');
  }
}
```

Network connection lost:

The SDK automatically attempts to reconnect. Listen for status changes:

javascript

```javascript
const stream = await DepotubeStream.start({
  // ...
  onStatusChange: (status) => {
    if (status === 'reconnecting') {
      showReconnectingIndicator();
    } else if (status === 'live') {
      hideReconnectingIndicator();
    } else if (status === 'error') {
      alert('Stream failed. Please check your internet connection.');
```

```
    }
  }
});
```

---

7. Automatic Transcription

7.1 Overview

depotube automatically transcribes all depositions using advanced speech-to-text AI. Transcription includes:

- **Speaker identification** - Distinguishes between attorney, witness, court reporter
- **Timestamps** - Every statement linked to video timecode
- **Punctuation and formatting** - Proper sentences and paragraphs
- **High accuracy** - Typical word error rate < 5%

Transcription begins automatically after:

- File upload completes (see Section 5)
- Live stream ends (see Section 6)

7.2 Transcription Timeline

Video Duration	Estimated Transcription Time
30 minutes	~8 minutes
1 hour	~15 minutes

2 hours	~30 minutes
4 hours	~60 minutes
6 hours	~90 minutes

Formula: Transcription time ≈ video duration ÷ 4

These are estimates. Actual time may vary based on:

- Audio quality
- Number of speakers
- Background noise
- Accents or technical terminology

7.3 Retrieve Transcript

Endpoint: `GET /api/v1/depositions/{deposition_id}/transcript`

Required Scope: `deposition:read`

Description: Retrieve the completed transcript for a deposition.

Request:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/transcript

Authorization: Bearer YOUR_ACCESS_TOKEN

Optional Query Parameters:

Parameter	Type	Description

`format`	string	Response format: `json` (default), `text`, `pdf`, `docx`
`include_timestamps`	boolean	Include timecodes (default: `true`)
`include_confidence`	boolean	Include word-level confidence scores (default: `false`)

Response (JSON format):

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "deposition_id": "dep_xyz789",
  "client_id": "client_abc123",
  "witness_name": "John Defendant",
  "deposition_date": "2026-02-01",
  "transcription_completed_at": "2026-02-01T10:48:00Z",
  "duration_seconds": 7200,
  "word_count": 42500,
  "speakers": [
    {
```

```
    "speaker_id": "speaker_1",
    "speaker_label": "Attorney",
    "identified_as": "Sarah Johnson"
  },
  {
    "speaker_id": "speaker_2",
    "speaker_label": "Witness",
    "identified_as": "John Defendant"
  },
  {
    "speaker_id": "speaker_3",
    "speaker_label": "Court Reporter",
    "identified_as": "Mary Smith"
  }
],
"transcript": [
  {
    "segment_id": 1,
    "speaker_id": "speaker_3",
    "speaker_label": "Court Reporter",
    "start_time": 0.0,
    "end_time": 4.2,
    "text": "Please state your full name for the record.",
```

```
  "confidence": 0.98
},
{
  "segment_id": 2,
  "speaker_id": "speaker_2",
  "speaker_label": "Witness",
  "start_time": 4.5,
  "end_time": 7.1,
  "text": "John Michael Defendant.",
  "confidence": 0.95
},
{
  "segment_id": 3,
  "speaker_id": "speaker_1",
  "speaker_label": "Attorney",
  "start_time": 8.0,
  "end_time": 12.3,
  "text": "Mr. Defendant, do you recall the events of January 15, 2026?",
  "confidence": 0.97
},
{
  "segment_id": 4,
  "speaker_id": "speaker_2",
```

```
    "speaker_label": "Witness",
    "start_time": 13.0,
    "end_time": 16.8,
    "text": "I think so, but I'm not entirely certain about the exact time.",
    "confidence": 0.92
  }
  // ... thousands more segments
 ]
}
```


Response Fields:

Field	Type	Description
`deposition_id`	string	Unique deposition identifier
`client_id`	string	Client this deposition belongs to
`witness_name`	string	Name of deposed person
`deposition_date`	ISO 8601 date	Date of deposition
`transcription_completed_at`	ISO 8601 datetime	When transcription finished

`duration_seconds`	integer	Total video duration
`word_count`	integer	Approximate word count
`speakers`	array	List of identified speakers
`transcript`	array	Array of transcript segments

Transcript Segment Fields:

Field	Type	Description
`segment_id`	integer	Sequential segment number
`speaker_id`	string	Speaker identifier
`speaker_label`	string	Role: "Attorney", "Witness", "Court Reporter"
`start_time`	float	Start time in seconds from video beginning
`end_time`	float	End time in seconds

`text`	string	Transcribed text for this segment
`confidence`	float	Transcription confidence (0.0 - 1.0)

7.4 Download Transcript (Alternative Formats)

Plain text format:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/transcript?format=text

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: text/plain

Content-Disposition: attachment; filename="deposition_dep_xyz789.txt"

Deposition of John Defendant

Case: CV-2026-12345

Date: February 1, 2026

Transcribed: February 1, 2026 10:48 AM

[00:00:00] Court Reporter: Please state your full name for the record.

[00:00:04] Witness: John Michael Defendant.

[00:00:08] Attorney: Mr. Defendant, do you recall the events of January 15, 2026?

[00:00:13] Witness: I think so, but I'm not entirely certain about the exact time.

...

PDF format:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/transcript?format=pdf

Authorization: Bearer YOUR_ACCESS_TOKEN

Returns a professionally formatted PDF with:

- Cover page with case information
- Table of contents
- Page numbers
- Timestamps in margin
- Speaker labels

Word document format:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/transcript?format=docx

Authorization: Bearer YOUR_ACCESS_TOKEN

Returns a Microsoft Word .docx file with editable text.

7.5 Search Transcript

Endpoint: `GET /api/v1/depositions/{deposition_id}/transcript/search`

Required Scope: `deposition:read`

Description: Search for specific text within a transcript.

Request:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/transcript/search?q=accident+reconstruction

Authorization: Bearer YOUR_ACCESS_TOKEN

Query Parameters:

Parameter	Type	Description
`q`	string	Search query (required)
`speaker`	string	Filter by speaker: "attorney", "witness", "court_reporter"
`limit`	integer	Max results to return (default: 10, max: 100)

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "query": "accident reconstruction",
  "total_matches": 7,
  "results": [
    {
      "segment_id": 456,
      "speaker_label": "Attorney",
      "start_time": 3240.5,
      "end_time": 3248.2,
      "text": "Did you speak with any accident reconstruction experts before today?",
      "match_context": "...Did you speak with any <mark>accident reconstruction</mark> experts before today?...",
      "video_timecode": "00:54:00"
    },
    {
      "segment_id": 789,
      "speaker_label": "Witness",
      "start_time": 4512.0,
      "end_time": 4520.5,
```

"text": "The police mentioned they would conduct an accident reconstruction analysis.",

"match_context": "...police mentioned they would conduct an <mark>accident reconstruction</mark> analysis...",

"video_timecode": "01:15:12"

}

]

}

7.6 Transcript Corrections

Endpoint: `PATCH /api/v1/depositions/{deposition_id}/transcript`

Required Scope: `attorney` or `admin`

Description: Correct transcription errors manually. Creates a new version while preserving the original.

Request:

http

PATCH https://api.depotube.com/v1/depositions/dep_xyz789/transcript

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: application/json

{

"corrections": [

{

"segment_id": 234,

```
    "original_text": "I was driving my Honda Civic",
    "corrected_text": "I was driving my Honda Accord",
    "reason": "Witness misspoke, corrected in later testimony"
  },
  {
    "segment_id": 567,
    "original_text": "around 3:00 PM",
    "corrected_text": "around 3:07 PM",
    "reason": "Cell phone records show 3:07 PM"
  }
 ]
}
```


Response:

http

```
HTTP/1.1 200 OK
Content-Type: application/json
```


```
{
  "deposition_id": "dep_xyz789",
  "corrections_applied": 2,
  "transcript_version": 2,
  "corrected_at": "2026-02-03T16:30:00Z",
  "corrected_by": "sarah.johnson@lawfirm.com"
```

}

Note: Original transcript is preserved as version 1. Corrected version is version 2.

To retrieve a specific version:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/transcript?version=1

---

8. AI Summary Generation

8.1 Overview

depotube AI summaries analyze deposition transcripts and generate structured legal analysis documents that include:

- Executive overview of key testimony
- Identified facts and admissions
- Credibility issues and contradictions
- Relevant case law citations in Bluebook format
- Cross-references to specific transcript locations

AI summaries save 6-9 hours of manual legal research per deposition.

8.2 Request AI Summary

Endpoint: `POST /api/v1/depositions/{deposition_id}/summarize`

Required Scope: `attorney` or `deposition:summarize`

Description: Initiate AI analysis and summary generation for a deposition. Requires a completed transcript.

Request:

http

POST https://api.depotube.com/v1/depositions/dep_xyz789/summarize

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: application/json

```
{
  "summary_type": "detailed",
  "focus_areas": ["liability", "damages", "credibility"],
  "include_case_law": true,
  "jurisdiction": "California",
  "case_type": "personal_injury"
}
```

Request Parameters:

Field	Type	Required	Description
`summary_type`	string	No	`brief` or `detailed` (default: `detailed`)
`focus_areas`	array of strings	No	Areas to emphasize: `liability`, `damages`, `credibility`, `causation`
`include_case_law`	boolean	No	Include case law citations (default: `true`)

`jurisdiction`	string	No	Relevant jurisdiction for case law (e.g., "California", "Federal")
`case_type`	string	No	Type of case: `personal_injury`, `contract`, `employment`, `medical_malpractice`, etc.

Response:

http

HTTP/1.1 202 Accepted

Content-Type: application/json


```
{
  "deposition_id": "dep_xyz789",
  "summary_job_id": "sum_job_456",
  "status": "processing",
  "created_at": "2026-02-03T15:00:00Z",
  "estimated_completion": "2026-02-03T15:10:00Z"
}
```


8.3 Poll Summary Status

Endpoint: `GET /api/v1/depositions/{deposition_id}/summary`

Required Scope: `deposition:read`

Description: Check the status of summary generation. Poll every 10 seconds.

Request:

http

```
GET https://api.depotube.com/v1/depositions/dep_xyz789/summary
Authorization: Bearer YOUR_ACCESS_TOKEN
```

Response (processing):

http

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "deposition_id": "dep_xyz789",
  "summary_job_id": "sum_job_456",
  "status": "processing",
  "progress_percent": 45,
  "current_step": "Searching case law database",
  "estimated_completion": "2026-02-03T15:10:00Z"
}
```

Response (complete):

http

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "deposition_id": "dep_xyz789",
  "summary_id": "sum_789",
  "status": "complete",
  "completed_at": "2026-02-03T15:09:00Z",
  "summary_url": "https://api.depotube.com/v1/depositions/dep_xyz789/summary"
}
```


8.4 Retrieve AI Summary

Once status is "complete", retrieve the full summary:

Request:

http

```
GET https://api.depotube.com/v1/depositions/dep_xyz789/summary
Authorization: Bearer YOUR_ACCESS_TOKEN
```

Response:

http

```
HTTP/1.1 200 OK
Content-Type: application/json
```


```
{
  "summary_id": "sum_789",
  "deposition_id": "dep_xyz789",
  "client_id": "client_abc123",
```

"witness_name": "John Defendant",

"deposition_date": "2026-02-01",

"generated_at": "2026-02-03T15:09:00Z",

"executive_overview": "The defendant testified under oath regarding the vehicular accident that occurred on January 15, 2026. Key issues emerged around timeline discrepancies and contradictions with physical evidence. The witness displayed significant uncertainty regarding critical facts, using hedging language 47 times throughout the deposition. Cell phone location data conflicts with the defendant's claimed whereabouts at the time of the incident.",

"key_facts_admitted": [

{

"fact": "Defendant confirmed ownership of 2018 Honda Accord (VIN: 1HGBH41JXMN109186)",

"transcript_reference": "Segment 45, 00:12:30",

"legal_significance": "Establishes vehicle ownership and control"

},

{

"fact": "Admitted to three prior speeding violations between 2024-2025",

"transcript_reference": "Segment 123, 00:34:15",

"legal_significance": "Pattern of traffic violations relevant to negligence claim"

},

{

"fact": "Could not recall date of last vehicle brake inspection",

"transcript_reference": "Segment 234, 00:56:20",
"legal_significance": "Potential maintenance negligence"
},
{
"fact": "Received traffic citation on January 10, 2026 (5 days before accident)",
"transcript_reference": "Segment 345, 01:15:45",
"legal_significance": "Temporal proximity suggests awareness of vehicle issues"
}
],

"contradictions_and_weaknesses": [
{
"issue": "Timeline contradiction regarding defendant's location",
"contradiction_details": {
"claim_1": "Testified he was 'at home' on January 15 at 3:00 PM",
"claim_1_reference": "Segment 456, 01:32:10",
"claim_2": "Cell phone records show device within 500 feet of accident scene at 3:07 PM",
"evidence_reference": "Exhibit C (cell tower data)"
},
"legal_impact": "Creates credibility issue and potential false statement under oath",
"impeachment_value": "High - direct contradiction with objective evidence"
},

```
{
  "issue": "Excessive hedging language undermines certainty",
  "contradiction_details": {
    "instances": 47,
    "examples": [
      "I think I was...",
      "Maybe around...",
      "I'm not sure but..."
    ]
  },
  "legal_impact": "Suggests unreliable memory or evasiveness",
  "impeachment_value": "Moderate - pattern undermines overall credibility"
},
{
  "issue": "Conflict with prior written declaration",
  "contradiction_details": {
    "claim_1": "Deposition: 'I had my vehicle inspected in December 2025'",
    "claim_1_reference": "Segment 567, 01:48:30",
    "claim_2": "Written declaration (March 2025): 'Last inspection was October 2024'",
    "evidence_reference": "Defendant's Declaration,    12"
  },
  "legal_impact": "Inconsistent statements about vehicle maintenance",
  "impeachment_value": "High - sworn statement conflicts with sworn testimony"
```

```
    }
  ],

  "case_law_citations": [
    {
      "case_name": "Daubert v. Merrell Dow Pharmaceuticals, Inc.",
      "citation": "509 U.S. 579 (1993)",
      "bluebook_citation": "Daubert v. Merrell Dow Pharm., Inc., 509 U.S. 579 (1993)",
      "relevance": "Standard for admissibility of expert testimony regarding cell phone location data",
      "holding": "Scientific evidence must meet reliability and relevance standards under Federal Rules of Evidence",
      "application_to_case": "Cell phone geolocation data is widely accepted as scientifically reliable evidence. Defendant's challenge to cell tower data must meet Daubert standard. Courts routinely admit such evidence in criminal and civil cases.",
      "key_quote": "The subject of an expert's testimony must be 'scientific knowledge.' The adjective 'scientific' implies a grounding in the methods and procedures of science.",
      "transcript_reference": "Segment 678, 02:03:15 (defendant questions validity of cell phone location data)",
      "footnote_number": 1
    },
    {
      "case_name": "Anderson v. Liberty Lobby, Inc.",
      "citation": "477 U.S. 242 (1986)",
      "bluebook_citation": "Anderson v. Liberty Lobby, Inc., 477 U.S. 242 (1986)",
```

"relevance": "Summary judgment standard when credibility is disputed",

"holding": "Contradictory testimony creates genuine issue of material fact precluding summary judgment",

"application_to_case": "Defendant's conflicting statements about his whereabouts on January 15 create a credibility dispute. This factual dispute must be resolved by jury, not summary judgment.",

"key_quote": "Credibility determinations, the weighing of the evidence, and the drawing of legitimate inferences from the facts are jury functions, not those of a judge.",

"transcript_reference": "Segment 456, 01:32:10 vs. Segment 789, 02:15:30",

"footnote_number": 2

},

{

"case_name": "Celotex Corp. v. Catrett",

"citation": "477 U.S. 317 (1987)",

"bluebook_citation": "Celotex Corp. v. Catrett, 477 U.S. 317 (1987)",

"relevance": "Burden of proof on summary judgment motions",

"holding": "Moving party bears burden of showing absence of genuine dispute of material fact",

"application_to_case": "Plaintiff has raised sufficient doubt regarding defendant's location to survive summary judgment. Cell phone data combined with contradictory testimony creates triable issue.",

"key_quote": "The moving party bears the burden of showing that there is no genuine issue as to any material fact.",

"transcript_reference": "Overall deposition contradictions",

"footnote_number": 3

},

{
"case_name": "People v. Superior Court (Granillo)",
"citation": "205 Cal. App. 4th 1404 (2012)",
"bluebook_citation": "People v. Superior Court (Granillo), 205 Cal. App. 4th 1404 (2012)",
"relevance": "California precedent on cell phone location evidence",
"holding": "Cell site location information is reliable evidence of a person's location",
"application_to_case": "Under California law, cell tower data showing defendant's phone near accident scene is admissible and probative evidence of defendant's physical presence.",
"key_quote": "Cell site location information can place a suspect in the vicinity of a crime.",
"transcript_reference": "Segment 678, 02:03:15",
"footnote_number": 4,
"jurisdiction_specific": true
}
],

"recommendations": [
"Conduct further discovery on defendant's cell phone records for January 15, 2026",
"Obtain defendant's vehicle maintenance records from Honda dealership",
"Depose cell phone data expert to establish foundation for Daubert hearing",
"File motion to compel production of defendant's complete phone records",
"Consider motion for sanctions regarding contradictory sworn statements"

```
  ],

  "summary_statistics": {
    "deposition_duration_minutes": 120,
    "transcript_word_count": 42500,
    "key_facts_identified": 4,
    "contradictions_identified": 3,
    "case_law_citations": 4,
    "hedging_language_instances": 47
  }
}
```


8.5 Download Summary (PDF Format)

Request:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/summary?format=pdf

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

Returns a professionally formatted PDF with:

- Law firm letterhead
- Case caption
- Table of contents
- Executive summary
- Detailed analysis sections
- Footnoted case law citations in Bluebook format

- Cross-references to transcript page/line numbers
- Appendix with full case citations

File naming: `Summary_Smith_John_2026-02-01.pdf`

8.6 Regenerate Summary

If you need to regenerate the summary with different parameters:

Request:

http

POST https://api.depotube.com/v1/depositions/dep_xyz789/summarize

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: application/json

```
{
  "summary_type": "brief",
  "focus_areas": ["damages"],
  "jurisdiction": "Federal",
  "regenerate": true
}
```

Setting `regenerate: true` creates a new summary, replacing the previous one.

---

9. Retrieving Depositions and Assets

9.1 List All Depositions for a Client

Endpoint: `GET /api/v1/clients/{client_id}/depositions`

Required Scope: `deposition:read`

Description: Retrieve all depositions within a client silo.

Request:

http

GET https://api.depotube.com/v1/clients/client_abc123/depositions

Authorization: Bearer YOUR_ACCESS_TOKEN

Optional Query Parameters:

Parameter	Type	Description
`status`	string	Filter by status: `uploading`, `transcribing`, `transcribed`, `summarized`, `failed`
`limit`	integer	Results per page (default: 50, max: 100)
`offset`	integer	Pagination offset
`sort`	string	Sort by: `deposition_date`, `created_at`, `witness_name`
`order`	string	`asc` or `desc`

Example:

http

GET https://api.depotube.com/v1/clients/client_abc123/depositions?status=transcribed&sort=deposition_date&order=desc

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "client_id": "client_abc123",
  "client_name": "Smith, John",
  "depositions": [
    {
      "deposition_id": "dep_xyz789",
      "witness_name": "John Defendant",
      "deposition_date": "2026-02-01",
      "status": "summarized",
      "created_at": "2026-02-01T10:15:00Z",
      "video_size_bytes": 4500000000,
      "duration_seconds": 7200,
      "transcript_available": true,
```

```
    "summary_available": true,
    "transcript_word_count": 42500
  },
  {
    "deposition_id": "dep_abc456",
    "witness_name": "Jane Witness",
    "deposition_date": "2026-01-25",
    "status": "transcribed",
    "created_at": "2026-01-25T14:30:00Z",
    "video_size_bytes": 3200000000,
    "duration_seconds": 5400,
    "transcript_available": true,
    "summary_available": false,
    "transcript_word_count": 31200
  }
],
"total": 2,
"limit": 50,
"offset": 0
}
```


9.2 Get Specific Deposition

Endpoint: `GET /api/v1/depositions/{deposition_id}`

Required Scope: `deposition:read`

Description: Retrieve detailed metadata for a single deposition.

Request:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "deposition_id": "dep_xyz789",
  "client_id": "client_abc123",
  "client_name": "Smith, John",
  "witness_name": "John Defendant",
  "deposition_date": "2026-02-01",
  "attorney_names": ["Sarah Johnson", "Michael Chen"],
  "case_number": "CV-2026-12345",
  "location": "San Francisco, CA",
  "status": "summarized",
  "created_at": "2026-02-01T10:15:00Z",
```

"updated_at": "2026-02-03T15:09:00Z",

"video": {
  "size_bytes": 4500000000,
  "duration_seconds": 7200,
  "format": "mp4",
  "resolution": "1920x1080",
  "bitrate_kbps": 5000,
  "framerate": 30,
  "codec": "h264",
  "url": "https://api.depotube.com/v1/depositions/dep_xyz789/video"
},

"transcript": {
  "available": true,
  "completed_at": "2026-02-01T10:48:00Z",
  "word_count": 42500,
  "version": 2,
  "corrections_applied": 2,
  "url": "https://api.depotube.com/v1/depositions/dep_xyz789/transcript"
},

"summary": {

```
    "available": true,
    "completed_at": "2026-02-03T15:09:00Z",
    "summary_id": "sum_789",
    "case_law_citations": 4,
    "contradictions_identified": 3,
    "url": "https://api.depotube.com/v1/depositions/dep_xyz789/summary"
  },

  "access_history": {
    "view_count": 15,
    "last_accessed_at": "2026-02-03T16:45:00Z",
    "last_accessed_by": "michael.chen@lawfirm.com"
  }
}
```


9.3 Download Video

Endpoint: `GET /api/v1/depositions/{deposition_id}/video`

Required Scope: `deposition:download`

Description: Download the original deposition video file.

Request:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/video

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: video/mp4

Content-Length: 4500000000

Content-Disposition: attachment; filename="deposition_Smith_John_2026-02-01.mp4"

[binary video data]

Alternative: Pre-signed URL (recommended for large files):

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/video?presigned=true

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "deposition_id": "dep_xyz789",
  "download_url": "https://storage.depotube.com/videos/dep_xyz789.mp4?signature=...",
  "expires_at": "2026-02-03T17:00:00Z",
  "size_bytes": 4500000000,
```

```
  "filename": "deposition_Smith_John_2026-02-01.mp4"
}
```

The `download_url` is valid for 1 hour and allows direct download without authentication.

9.4 Stream Video

Endpoint: `GET /api/v1/depositions/{deposition_id}/video/stream`

Required Scope: `deposition:read`

Description: Stream video for in-browser playback (supports range requests).

Request:

http

```
GET https://api.depotube.com/v1/depositions/dep_xyz789/video/stream
Authorization: Bearer YOUR_ACCESS_TOKEN
Range: bytes=0-1048575
```

Response:

http

```
HTTP/1.1 206 Partial Content
Content-Type: video/mp4
Content-Length: 1048576
Content-Range: bytes 0-1048575/4500000000
Accept-Ranges: bytes

[video chunk]
```

HTML5 Video Player Example:

html

```html
<video controls width="960" height="540">
  <source
    src="https://api.depotube.com/v1/depositions/dep_xyz789/video/stream"
    type="video/mp4">
</video>


<script>
  // Add auth token to video requests
  const video = document.querySelector('video');
  const source = video.querySelector('source');

  fetch(source.src, {
    headers: {
      'Authorization': 'Bearer YOUR_ACCESS_TOKEN'
    }
  })
  .then(response => response.blob())
  .then(blob => {
    source.src = URL.createObjectURL(blob);
    video.load();
  });
```

</script>

9.5 Download Assets as ZIP

Endpoint: `GET /api/v1/depositions/{deposition_id}/download-all`

Required Scope: `deposition:download`

Description: Download video, transcript, and summary as a single ZIP file.

Request:

http

GET https://api.depotube.com/v1/depositions/dep_xyz789/download-all

Authorization: Bearer YOUR_ACCESS_TOKEN

Optional Query Parameters:

Parameter	Type	Description
`include_video`	boolean	Include video file (default: `true`)
`transcript_format`	string	Transcript format: `json`, `text`, `pdf`, `docx` (default: `pdf`)
`summary_format`	string	Summary format: `json`, `pdf` (default: `pdf`)

Response:

http

HTTP/1.1 200 OK

Content-Type: application/zip

Content-Length: 4505000000

Content-Disposition: attachment; filename="deposition_dep_xyz789.zip"

[ZIP file containing:]

- deposition_Smith_John_2026-02-01.mp4

- transcript_Smith_John_2026-02-01.pdf

- summary_Smith_John_2026-02-01.pdf

---

10. Delete Operations

10.1 Delete Deposition

Endpoint: `DELETE /api/v1/depositions/{deposition_id}`

Required Scope: `admin` or `deposition:delete`

Description: Delete a deposition and all associated assets. Behavior depends on deposition status.

Request:

http

DELETE https://api.depotube.com/v1/depositions/dep_xyz789?confirm=true

Authorization: Bearer YOUR_ACCESS_TOKEN

Query Parameters:

Parameter	Type	Required	Description
`confirm`	boolean	Yes	Must be `true` to execute deletion

Deletion Behavior by Status:

Status	Behavior
`uploading`	Cancel upload, discard partial file
`transcribing`	Stop transcription, keep uploaded video, mark incomplete
`transcribed`	Delete video, transcript, all metadata
`summarized`	Delete video, transcript, summary, all metadata
`failed`	Delete any partial assets and metadata

Response (successful deletion):

http

```
HTTP/1.1 200 OK
```

Content-Type: application/json


```
{
  "deposition_id": "dep_xyz789",
  "deleted": true,
  "deleted_at": "2026-02-03T17:00:00Z",
  "deleted_by": "admin@lawfirm.com",
  "assets_deleted": {
    "video": true,
    "transcript": true,
    "summary": true
  },
  "bytes_freed": 4500000000,
  "audit_log_id": "audit_98765"
}
```


Response (missing confirmation):

http

HTTP/1.1 400 Bad Request

Content-Type: application/json


```
{
  "error": "confirmation_required",
  "error_description": "Add '?confirm=true' to delete this deposition",
```

```
  "deposition_id": "dep_xyz789",
  "assets_to_delete": {
    "video_size_bytes": 4500000000,
    "transcript_available": true,
    "summary_available": true
  }
}
```


Response (insufficient permissions):

http

HTTP/1.1 403 Forbidden

Content-Type: application/json


```
{
  "error": "insufficient_scope",
  "error_description": "User role 'paralegal' cannot delete depositions. Admin role required."
}
```


10.2 Cancel In-Progress Upload

Endpoint: `DELETE /api/v1/depositions/{deposition_id}`

Required Scope: `attorney` or `deposition:delete`

Description: Cancel an upload that is currently in progress.

Request:

http

DELETE https://api.depotube.com/v1/depositions/dep_xyz789?confirm=true

Authorization: Bearer YOUR_ACCESS_TOKEN

Response (upload cancelled):

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "deposition_id": "dep_xyz789",
  "status": "cancelled",
  "upload_percent": 45,
  "cancelled_at": "2026-02-01T10:20:00Z",
  "bytes_uploaded": 2025000000,
  "bytes_discarded": 2025000000,
  "deleted": true
}
```


10.3 Delete Client (Cascade)

Endpoint: `DELETE /api/v1/clients/{client_id}`

Required Scope: `admin` or `client:delete`

Description: Delete an entire client silo including all depositions. This is a destructive operation that cannot be undone.

Request:

http

DELETE https://api.depotube.com/v1/clients/client_abc123?confirm=true

Authorization: Bearer YOUR_ACCESS_TOKEN

Query Parameters:

Parameter	Type	Required	Description
`confirm`	boolean	Yes	Must be `true` to execute deletion

Response (successful deletion):

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "client_id": "client_abc123",
  "client_name": "Smith, John",
  "deleted": true,
  "deleted_at": "2026-02-03T17:30:00Z",
  "deleted_by": "admin@lawfirm.com",
  "depositions_deleted": 3,
```

```
  "total_bytes_freed": 12000000000,
  "assets_deleted": {
    "videos": 3,
    "transcripts": 3,
    "summaries": 2
  },
  "audit_log_id": "audit_98766"
}
```


Response (missing confirmation):

http


```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
  "error": "confirmation_required",
  "error_description": "This will permanently delete client 'Smith, John' and all 3 depositions. Add '?confirm=true' to proceed.",
  "client_id": "client_abc123",
  "client_name": "Smith, John",
  "deposition_count": 3,
  "total_storage_bytes": 12000000000
}
```

10.4 Bulk Delete Depositions

Endpoint: `POST /api/v1/depositions/bulk-delete`

Required Scope: `admin`

Description: Delete multiple depositions in a single request.

Request:

http

POST https://api.depotube.com/v1/depositions/bulk-delete

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: application/json

```
{
  "deposition_ids": [
    "dep_xyz789",
    "dep_abc456",
    "dep_def123"
  ],
  "confirm": true
}
```

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "requested": 3,
  "deleted": 3,
  "failed": 0,
  "deleted_at": "2026-02-03T17:45:00Z",
  "total_bytes_freed": 13500000000,
  "results": [
    {
      "deposition_id": "dep_xyz789",
      "deleted": true
    },
    {
      "deposition_id": "dep_abc456",
      "deleted": true
    },
    {
      "deposition_id": "dep_def123",
      "deleted": true
    }
  ]
}
```


---

11. Audit Logs

11.1 Overview

depotube maintains comprehensive audit logs for compliance and security purposes. All actions (create, read, update, delete) are logged with:

- User who performed the action
- Timestamp
- Resource affected (client, deposition, transcript, summary)
- Action type (created, viewed, downloaded, modified, deleted)
- IP address and user agent

Audit logs are immutable and retained for 7 years per legal industry standards.

11.2 Retrieve Audit Logs

Endpoint: `GET /api/v1/audit-logs`

Required Scope: `audit:read`

Description: Retrieve audit log entries. Admins see all logs; attorneys/paralegals see logs only for clients they have access to.

Request:

http

GET https://api.depotube.com/v1/audit-logs

Authorization: Bearer YOUR_ACCESS_TOKEN

Optional Query Parameters:

Parameter	Type	Description

`client_id`	string	Filter by client
`deposition_id`	string	Filter by deposition
`user_email`	string	Filter by user who performed action
`action_type`	string	Filter by action: `created`, `viewed`, `downloaded`, `modified`, `deleted`
`resource_type`	string	Filter by resource: `client`, `deposition`, `transcript`, `summary`
`start_date`	ISO 8601 date	Filter logs after this date
`end_date`	ISO 8601 date	Filter logs before this date
`limit`	integer	Results per page (default: 50, max: 100)
`offset`	integer	Pagination offset

Example:

http

GET
https://api.depotube.com/v1/audit-logs?client_id=client_abc123&action_type=deleted&start_date=2026-02-01

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json


```
{
  "audit_logs": [
    {
      "log_id": "audit_98765",
      "timestamp": "2026-02-03T17:00:00Z",
      "user_email": "admin@lawfirm.com",
      "user_role": "admin",
      "action_type": "deleted",
      "resource_type": "deposition",
      "resource_id": "dep_xyz789",
      "client_id": "client_abc123",
      "details": {
        "deposition_witness": "John Defendant",
        "assets_deleted": ["video", "transcript", "summary"],
```

"bytes_freed": 4500000000
},
"ip_address": "203.0.113.42",
"user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36"
},
{
"log_id": "audit_98764",
"timestamp": "2026-02-03T16:45:00Z",
"user_email": "sarah.johnson@lawfirm.com",
"user_role": "attorney",
"action_type": "downloaded",
"resource_type": "transcript",
"resource_id": "dep_xyz789",
"client_id": "client_abc123",
"details": {
"format": "pdf",
"file_size_bytes": 245000
},
"ip_address": "203.0.113.15",
"user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
},
{

```
      "log_id": "audit_98763",
      "timestamp": "2026-02-03T15:09:00Z",
      "user_email": "system",
      "user_role": "system",
      "action_type": "created",
      "resource_type": "summary",
      "resource_id": "sum_789",
      "client_id": "client_abc123",
      "details": {
        "deposition_id": "dep_xyz789",
        "case_law_citations": 4,
        "processing_time_seconds": 540
      },
      "ip_address": "internal",
      "user_agent": "depotube-ai-service/v1.2.3"
    }
  ],
  "total": 3,
  "limit": 50,
  "offset": 0
}
```


11.3 Export Audit Logs

Endpoint: `GET /api/v1/audit-logs/export`

Required Scope: `admin` or `audit:read`

Description: Export audit logs as CSV for compliance reporting.

Request:

http

GET https://api.depotube.com/v1/audit-logs/export?client_id=client_abc123&start_date=2026-01-01&end_date=2026-02-03

Authorization: Bearer YOUR_ACCESS_TOKEN

Response:

http

HTTP/1.1 200 OK

Content-Type: text/csv

Content-Disposition: attachment; filename="audit_logs_2026-01-01_2026-02-03.csv"

log_id,timestamp,user_email,user_role,action_type,resource_type,resource_id,client_id,ip_address

audit_98765,2026-02-03T17:00:00Z,admin@lawfirm.com,admin,deleted,deposition,dep_xyz789,client_abc123,203.0.113.42

audit_98764,2026-02-03T16:45:00Z,sarah.johnson@lawfirm.com,attorney,downloaded,transcript,dep_xyz789,client_abc123,203.0.113.15

...

11.4 Audit Log Retention

- **Retention period:** 7 years (2,555 days) from log creation

- **Immutable:** Audit logs cannot be modified or deleted by users
- **Encrypted:** Logs are encrypted at rest and in transit
- **Compliant with:** SOC 2, HIPAA, attorney-client privilege requirements

---

12. Error Handling

12.1 Standard Error Response Format

All API errors return a consistent JSON structure:

```
json
{
  "error": "error_code",
  "error_description": "Human-readable description of the error",
  "error_details": {
    "field": "specific_field_with_error",
    "value": "invalid_value_provided"
  },
  "request_id": "req_abc123",
  "timestamp": "2026-02-03T18:00:00Z"
}
```

12.2 HTTP Status Codes

Status Code	Meaning	When Used

200 OK	Success	Successful GET, PATCH, DELETE
201 Created	Resource created	Successful POST (client, deposition, summary)
202 Accepted	Request accepted, processing async	Summary generation initiated
204 No Content	Success, no response body	Successful DELETE with no return data
400 Bad Request	Invalid request	Missing required fields, invalid format
401 Unauthorized	Authentication failed	Missing or invalid access token
403 Forbidden	Insufficient permissions	Token lacks required scope
404 Not Found	Resource doesn't exist	Invalid deposition_id, client_id
409 Conflict	Resource conflict	Duplicate client, concurrent update
413 Payload Too Large	File too large	Video exceeds 100GB limit

415 Unsupported Media Type	Invalid file format	AVI file uploaded instead of MP4
422 Unprocessable Entity	Validation error	Invalid metadata values
429 Too Many Requests	Rate limit exceeded	Too many requests in time window
500 Internal Server Error	Server error	Unexpected server failure
502 Bad Gateway	Upstream service failure	Transcription service unavailable
503 Service Unavailable	Temporary outage	Scheduled maintenance

12.3 Common Error Examples

Authentication Errors

Missing access token:

http

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json

{
```

```
  "error": "missing_token",
  "error_description": "Authorization header is required",
  "request_id": "req_abc123"
}
```

Expired token:

```http
HTTP/1.1 401 Unauthorized
Content-Type: application/json

{
  "error": "expired_token",
  "error_description": "Access token expired at 2026-02-03T16:00:00Z",
  "request_id": "req_abc124"
}
```

Invalid token:

```http
HTTP/1.1 401 Unauthorized
Content-Type: application/json

{
  "error": "invalid_token",
  "error_description": "Access token is malformed or invalid",
```

```
  "request_id": "req_abc125"
}
```

Authorization Errors

Insufficient scope:

http

```
HTTP/1.1 403 Forbidden
Content-Type: application/json

{
  "error": "insufficient_scope",
  "error_description": "Token does not have 'client:delete' scope",
  "required_scope": "client:delete",
  "token_scopes": ["attorney", "deposition:read", "deposition:upload"],
  "request_id": "req_abc126"
}
```

No access to client:

http

```
HTTP/1.1 403 Forbidden
Content-Type: application/json

{
  "error": "access_denied",
```

"error_description": "User does not have access to client 'client_abc123'",

"client_id": "client_abc123",

"request_id": "req_abc127"

}

Resource Not Found

http

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "resource_not_found",

"error_description": "Deposition 'dep_xyz999' does not exist",

"resource_type": "deposition",

"resource_id": "dep_xyz999",

"request_id": "req_abc128"

}

Validation Errors

Missing required field:

http

HTTP/1.1 400 Bad Request

Content-Type: application/json

```
{
  "error": "validation_error",
  "error_description": "Required field 'witness_name' is missing",
  "error_details": {
    "field": "witness_name",
    "constraint": "required"
  },
  "request_id": "req_abc129"
}
```


Invalid field value:

http

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json
```


```
{
  "error": "validation_error",
  "error_description": "Invalid date format for 'deposition_date'",
  "error_details": {
    "field": "deposition_date",
    "value": "02/01/2026",
    "expected_format": "ISO 8601 (YYYY-MM-DD)",
    "example": "2026-02-01"
  },
```

```
  "request_id": "req_abc130"
}
```

File Upload Errors

File too large:

http

```
HTTP/1.1 413 Payload Too Large
Content-Type: application/json

{
  "error": "file_too_large",
  "error_description": "Video file exceeds maximum size of 100GB",
  "file_size_bytes": 120000000000,
  "max_size_bytes": 100000000000,
  "request_id": "req_abc131"
}
```

Unsupported format:

http

```
HTTP/1.1 415 Unsupported Media Type
Content-Type: application/json

{
  "error": "unsupported_format",
```

```
  "error_description": "Video format 'avi' is not supported",
  "file_format": "avi",
  "supported_formats": ["mp4", "mov", "webm"],
  "request_id": "req_abc132"
}
```


Rate Limiting

http


```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 3600
```

```
{
  "error": "rate_limit_exceeded",
  "error_description": "Too many requests. Limit is 100 requests per hour.",
  "rate_limit": {
    "limit": 100,
    "remaining": 0,
    "reset_at": "2026-02-03T19:00:00Z"
  },
  "request_id": "req_abc133"
}
```


Service Errors

Transcription service unavailable:

http

```
HTTP/1.1 502 Bad Gateway
Content-Type: application/json

{
  "error": "service_unavailable",
  "error_description": "Transcription service is temporarily unavailable",
  "service": "transcription",
  "estimated_recovery": "2026-02-03T18:30:00Z",
  "request_id": "req_abc134"
}
```

Maintenance mode:

http

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/json
Retry-After: 7200

{
  "error": "maintenance",
  "error_description": "depotube is undergoing scheduled maintenance",
  "maintenance_window": {
    "start": "2026-02-03T18:00:00Z",
```

```
    "end": "2026-02-03T20:00:00Z"
  },
  "request_id": "req_abc135"
}
```


12.4 Rate Limiting

depotube enforces rate limits to ensure fair usage and system stability:

Endpoint Type	Rate Limit	Window
Authentication	10 requests	Per minute
Upload/Stream	5 concurrent uploads	Per user
API reads (GET)	1000 requests	Per hour
API writes (POST/PATCH/DELETE)	100 requests	Per hour
Summary generation	10 requests	Per hour

Rate limit headers are included in all responses:

http

X-RateLimit-Limit: 1000

X-RateLimit-Remaining: 847

X-RateLimit-Reset: 1738608000

When rate limit is exceeded:

http

HTTP/1.1 429 Too Many Requests

X-RateLimit-Limit: 1000

X-RateLimit-Remaining: 0

X-RateLimit-Reset: 1738608000

Retry-After: 3600

Wait until the `X-RateLimit-Reset` timestamp before retrying.

12.5 Error Handling Best Practices

JavaScript example:

javascript

```javascript
async function callDepotubeAPI(endpoint, options) {
  try {
    const response = await fetch(endpoint, options);


    if (!response.ok) {
      const error = await response.json();


      // Handle specific error types
      if (error.error === 'expired_token') {
```

```
      // Refresh token and retry
      await refreshAccessToken();
      return callDepotubeAPI(endpoint, options);
    } else if (error.error === 'rate_limit_exceeded') {
      // Wait and retry
      const retryAfter = response.headers.get('Retry-After');
      await sleep(retryAfter * 1000);
      return callDepotubeAPI(endpoint, options);
    } else if (response.status === 404) {
      console.error('Resource not found:', error.error_description);
      return null;
    } else {
      console.error('API error:', error.error_description);
      throw new Error(error.error_description);
    }
  }

  return await response.json();

} catch (networkError) {
  console.error('Network error:', networkError);
  throw networkError;
}
```

}

---

13. Appendix

13.1 Video Format Specifications

Supported Formats

Container	Video Codec	Audio Codec	Extension	Max Size
MPEG-4	H.264 (AVC)	AAC	.mp4	100 GB
QuickTime	H.264 (AVC)	AAC	.mov	100 GB
WebM	VP8, VP9	Vorbis, Opus	.webm	100 GB

Resolution and Quality

Quality Level	Resolution	Bitrate (Recommended)	Use Case
Minimum	480p (640×480)	1 Mbps	Low bandwidth upload

Standard	720p (1280×720)	2-3 Mbps	Default for most depositions
High	1080p (1920×1080)	5 Mbps	Professional recording
Maximum	4K (3840×2160)	20-50 Mbps	High-detail evidence

Audio Requirements

- **Sample rate:** 44.1 kHz or 48 kHz
- **Bit depth:** 16-bit minimum
- **Channels:** Mono or stereo
- **Codec:** AAC (preferred), Vorbis, Opus

Frame Rate

- **Minimum:** 24 fps
- **Standard:** 30 fps
- **Maximum:** 60 fps

13.2 Transcription Estimates

Processing Time

Video Duration	Estimated Transcription Time	Formula
10 minutes	~2.5 minutes	Duration ÷ 4

30 minutes	~8 minutes	Duration ÷ 4
1 hour	~15 minutes	Duration ÷ 4
2 hours	~30 minutes	Duration ÷ 4
4 hours	~60 minutes	Duration ÷ 4
6 hours	~90 minutes	Duration ÷ 4
8 hours	~120 minutes	Duration ÷ 4

Note: Times are estimates. Actual processing may vary based on audio quality, number of speakers, background noise, and accents.

Transcription Accuracy

- **Typical word error rate:** < 5%
- **Confidence threshold:** 0.85 (85%)
- **Speaker identification accuracy:** > 95% when speakers have distinct voices

Factors Affecting Accuracy

Factor	Impact	Mitigation

Background noise	Decreased accuracy	Use quiet environment, noise-canceling microphone
Multiple overlapping speakers	Difficult to separate	Encourage one speaker at a time
Heavy accents	Slightly decreased accuracy	AI adapts over time with more data
Technical jargon	May require corrections	Manual review and correction feature available
Poor audio quality	Significantly decreased	Use external microphone, minimum 44.1 kHz sample rate

13.3 AI Summary Processing Time

Transcript Length	Summary Generation Time
< 10,000 words (< 1 hour deposition)	~5 minutes
10,000-30,000 words (1-2 hour)	~8 minutes
30,000-50,000 words (2-4 hour)	~12 minutes

50,000+ words (4+ hour)	~15-20 minutes

Processing steps:

1. Transcript analysis: 30-40% of time
2. Case law database search: 40-50% of time
3. Citation formatting and generation: 10-20% of time

13.4 Storage Estimates

Video File Sizes

Duration	Resolution	Bitrate	Approximate Size
1 hour	720p	2 Mbps	~900 MB
1 hour	1080p	5 Mbps	~2.2 GB
2 hours	720p	2 Mbps	~1.8 GB
2 hours	1080p	5 Mbps	~4.5 GB
4 hours	720p	2 Mbps	~3.6 GB

4 hours	1080p	5 Mbps	~9 GB
8 hours	720p	2 Mbps	~7.2 GB
8 hours	1080p	5 Mbps	~18 GB

Transcript File Sizes

Format	Size (per 1 hour deposition)
JSON	~500 KB
Plain text	~150 KB
PDF	~200 KB
DOCX	~180 KB

AI Summary File Sizes

Format	Size

JSON	~50-100 KB
PDF	~300-500 KB

13.5 Webhook Configuration

depotube supports webhooks for asynchronous notifications.

Available Events

Event	Description	Payload
`deposition.upload.complete`	Upload finished	`deposition_id`, `status`
`deposition.transcription.complete`	Transcript ready	`deposition_id`, `transcript_url`
`deposition.summary.complete`	Summary ready	`deposition_id`, `summary_id`, `summary_url`
`deposition.deleted`	Deposition deleted	`deposition_id`, `deleted_at`
`client.created`	New client created	`client_id`, `name`

`client.deleted`	Client deleted	`client_id`, `depositions_deleted`

Configure Webhook Endpoint

Endpoint: `POST /api/v1/webhooks`

Request:

http

POST https://api.depotube.com/v1/webhooks

Authorization: Bearer YOUR_ACCESS_TOKEN

Content-Type: application/json

```
{
  "url": "https://yourapp.com/webhooks/depotube",
  "events": [
    "deposition.transcription.complete",
    "deposition.summary.complete"
  ],
  "secret": "your_webhook_secret"
}
```

Response:

json

```
{
```

```
  "webhook_id": "webhook_123",
  "url": "https://yourapp.com/webhooks/depotube",
  "events": ["deposition.transcription.complete", "deposition.summary.complete"],
  "created_at": "2026-02-03T18:00:00Z",
  "active": true
}
```

Webhook Payload Example

When transcription completes, depotube sends:

```http
POST https://yourapp.com/webhooks/depotube
Content-Type: application/json
X-Depotube-Signature: sha256=abc123...
{
  "event": "deposition.transcription.complete",
  "timestamp": "2026-02-03T15:30:00Z",
  "deposition_id": "dep_xyz789",
  "client_id": "client_abc123",
  "transcript_url": "https://api.depotube.com/v1/depositions/dep_xyz789/transcript",
  "transcript_word_count": 42500
}
```

Verify Webhook Signature

javascript

```
const crypto = require('crypto');
function verifyWebhook(payload, signature, secret) {
  const expectedSignature = crypto
    .createHmac('sha256', secret)
    .update(JSON.stringify(payload))
    .digest('hex');
  return `sha256=${expectedSignature}` === signature;
}
// In your webhook handler
app.post('/webhooks/depotube', (req, res) => {
  const signature = req.headers['x-depotube-signature'];
  const isValid = verifyWebhook(req.body, signature, 'your_webhook_secret');

  if (!isValid) {
    return res.status(401).send('Invalid signature');
  }
  // Process webhook event
  const event = req.body.event;
  if (event === 'deposition.transcription.complete') {
    console.log('Transcript ready:', req.body.deposition_id);
  }
  res.status(200).send('OK');
});
```

13.6 API Versioning

- **Current version:** `v1`
- **Base URL:** `https://api.depotube.com/v1`
- **Version in URL path:** All endpoints include version number
- **Breaking changes:** Will result in new version (e.g., `v2`)
- **Deprecation period:** Minimum 12 months notice for version sunset

13.7 SDK and Library Support

Official SDKs:

- JavaScript/TypeScript: `npm install @depotube/sdk`
- Python: `pip install depotube`
- Ruby: `gem install depotube`

Community SDKs:

- PHP: `composer require depotube/php-sdk`
- Java: Maven/Gradle packages available

13.8 Support and Contact

- **API Documentation:** https://docs.depotube.com
- **Support Email:** support@depotube.com
- **Status Page:** https://status.depotube.com
- **Developer Forum:** https://forum.depotube.com
- **GitHub Issues:** https://github.com/depotube/api/issues